



Tecnicatura Universitaria
en Programación

PROGRAMACIÓN III

FRONT END

Unidad Temática N° 3:
Angular

Teórico
2° Año – 3° Cuatrimestre



Índice

Desarrollo Web Actual y Angular	8
Introducción al Desarrollo Front-End Moderno y las Single Page Applications (SPA)	8
¿Por qué SPA?	8
Diagrama: Arquitectura de una SPA	9
Angular: Un Framework Integral para el Desarrollo de Aplicaciones Web Modernas	9
Historia y Evolución: De AngularJS a Angular Moderno	10
(Angular 1): Los Inicios (2010)	10
La Gran Reescritura: El Nacimiento de Angular Moderno (Desde Angular 2 en adelante, 2016)	10
Características Clave de Angular	11
Actualización 2026: Angular 20 y Angular 21 - La Consolidación de un Framework Moderno	12
Angular v20 (Mayo 2025): Eficiencia y Estándares de Rendimiento	12
Angular v21 (Noviembre 2025): Testing, Accesibilidad y Modernización de Herramientas	13
Conclusión: Un Framework Preparado para el Futuro	13
Herramientas necesarias para el desarrollo con Angular	14
Node.js	14
Documentación	14
El entorno de desarrollo (IDE)	15
Conocimientos base requeridos	15
Errores comunes y diagnóstico	15
Angular CLI	15
Documentación	16
Configuración Angular CLI	16
Comandos	17
Errores comunes y diagnóstico	17
Actualización 2026: ng update y migración automática	18
Creando nuestro primer proyecto de Angular	18
¿Habilitar Server-Side Rendering (SSR)?	18

¿Qué formato de estilos utilizar?	18
Primer proyecto: comandos y estructura	19
Estructura del proyecto	19
Árbol de archivos principal	21
Comandos de ejecución	21
Componentes y Standalone Components	22
¿Qué son los Componentes en Angular?	22
¿Para qué sirven los componentes?	22
Creación de un Componente	22
Componentes Tradicionales (con Módulos)	22
Componentes Autónomos (Standalone Components)	22
Estructura de un Componente	23
Componente Tradicional	23
Componente Autónomo (Standalone)	24
Uso de un Componente	24
Componentes Tradicionales	24
Componentes Autónomos (Standalone)	24
Ventajas de los Standalone Components (Angular 17+)	25
Inyección de Dependencias en Standalone Components	26
¿Cuándo usar Standalone Components?	26
Conclusión	26
Errores comunes y diagnóstico	26
Ciclo de Vida de Componentes	27
Eventos Clave del Ciclo de Vida (Hooks)	27
2. Contenido Proyectado (Content Projection)	28
3. Vista del Componente (Component View)	29
4. Destrucción	29
Flujo de Ejecución Típico	29

Binding	30
One way binding	30
Two way binding	31
Resumen comparativo de Data Binding en Angular	32
Errores comunes y diagnóstico	32
Directivas	32
Directivas estructurales	33
Directivas de atributos	34
Directivas personalizadas.....	34
Código directiva:	35
Utilización directiva:	35
Otras directivas	35
ng-content.....	35
ng-template	36
ng-container.....	37
Nuevas Directivas Estructurales en Angular 18: @if, @else, @for y @switch	38
@if y @else: Renderizado Condicional Mejorado y Simplificado	38
La directiva @if.....	38
La directiva @for.....	39
@switch y @case: Una Directiva Estructural Moderna y Expresiva	41
Pipes en Angular: Transformación de Datos en las Plantillas	44
Pipes de Transformación de Cadenas de Texto	44
Pipes de Fechas y Horas.....	45
Pipes Numéricos y Financieros	45
Pipes de Manipulación de Colecciones y Subcadenas.....	46
Pipes Personalizados (Custom Pipes)	46
Creación de un Pipe Personalizado:	46
Routing en Angular: Navegación Potente en Single Page Applications (SPA)	48

Implementación y Configuración Detallada del Routing.....	49
Paso 1: Definición de la Tabla de Rutas (app.routes.ts):.....	49
Paso 2: Registro de las Rutas (app.config.ts):	49
Paso 3: Inserción del router-outlet (app.component.html):	50
Rutas Dinámicas y Parámetros de Ruta	50
Sintaxis de Ruta Dinámica:	50
Acceso a Parámetros con withRouter() (Recomendado Angular 16+):.....	50
Lazy Loading (Carga Diferida).....	51
Guards (Protección de Rutas).....	51
Navegación Programática (Imperativa).....	52
Opciones Avanzadas del Router (withFeatures).....	53
Diagnóstico de Errores Comunes y Troubleshooting	53
Buenas Prácticas de Routing en Angular 2026.....	54
Comunicación entre Componentes en Angular	55
1. Comunicación de Padre a Hijo con @Input en Angular	55
Flujo y Mecanismo:.....	55
Pasos Clave:.....	56
Diagrama:	56
2. Comunicación de Hijo a Padre: @Output y EventEmitter	56
Principios Clave y Beneficios	57
Flujo Detallado y Mecanismos de Angular	57
Comunicación de Hijo a Padre (Emisión de Eventos).....	57
Implementación Paso a Paso con Código.....	57
Diagrama de Flujo Conceptual (Hijo → Padre)	59
Flujo completo Padre ↔ Hijo.....	59
Servicios (Alternativa para Comunicación).....	60
3. Comunicación entre componentes “hermanos” (Servicios)	60
Errores comunes y diagnóstico	62

Servicios e Inyección de Dependencias	63
Servicios	63
Características:	63
Ejemplo:	64
Errores comunes y diagnóstico	68
Reactividad con RxJS, AsyncPipe, HTTP y Formularios Reactivos	68
Introducción	68
1. El paradigma de la programación reactiva y RxJS	69
El concepto central: Observable	69
¿Qué es?	69
Características claves:	69
Usos más comunes:	70
Modelo mental de un flujo reactivo	70
Ventaja estratégica en aplicaciones modernas	71
Anatomía de un Observable y ciclo de vida del stream	72
1. Creación	72
2. Suscripción	73
3. Ejecución	73
4. Finalización o cancelación	74
Gestión moderna de suscripciones en Angular 2026	74
Ejemplo	75
Diagrama del ciclo completo	75
Observables finitos vs infinitos	76
Observables finitos	76
Observables infinitos	76
Buenas prácticas actuales	77
Operadores RxJS: el motor de la transformación	77
Categorización de operadores esenciales	77

Operadores de aplanamiento: un punto clave	78
HttpClient	79
Configuración.....	80
Funcionalidades configurables	80
withFetch()	80
withInterceptors([...])	80
withInterceptorsFromDi()	80
withJsonpSupport().....	80
Ejemplo conceptual de petición	82
Buenas prácticas actuales	83
Errores comunes y diagnóstico	84
Formularios en Angular	84
Tipos de Formularios en Angular: Enfoques para el Desarrollo.....	86
1. Template-Driven Forms (Formularios Basados en Plantillas).....	86
2. Reactive Forms: La Arquitectura Model-Driven de Angular	88
El Paradigma Reactivo: La Sinergia con RxJS	89
Gestión de Validaciones: Sincrónicas y Asíncronas.....	90
Validaciones Sincrónicas (Synchronous Validators)	90
Validaciones Asíncronas (Asynchronous Validators).....	90
Piezas Fundamentales de la Arquitectura Reactiva (La Jerarquía del Modelo)	91
Flujo de Trabajo Detallado (Model-Driven).....	92
1. Configuración del Módulo	92
2. Conexión al Template (HTML - El "Binding")	92
Ventajas Definitivas del Enfoque Reactivo.....	93
Resumen y Checklist	94

Desarrollo Web Actual y Angular

Introducción al Desarrollo Front-End Moderno y las Single Page Applications (SPA)

En la última década, el desarrollo web ha evolucionado significativamente, impulsado por la creciente demanda de aplicaciones más interactivas, rápidas y con una experiencia de usuario fluida. En el pasado, la mayoría de las aplicaciones web seguían un modelo de servidor tradicional, donde cada interacción del usuario requería una nueva carga de página completa desde el servidor. Este enfoque, aunque funcional, tenía limitaciones notables en términos de rendimiento y experiencia de usuario.

Hoy en día, el desarrollo front-end se centra en ofrecer aplicaciones ricas e interactivas que se comportan de manera más parecida a las aplicaciones de escritorio, brindando una experiencia más dinámica y reactiva. Este cambio ha sido posible gracias al surgimiento de las Single Page Applications (SPA).

¿Por qué SPA?

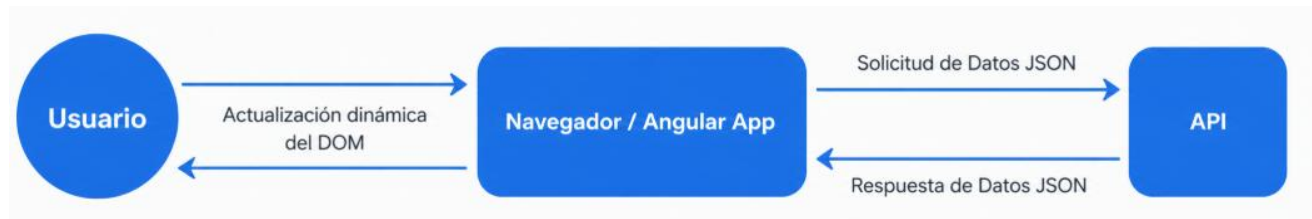
Las **SPA** son aplicaciones web que cargan una única página HTML y actualizan dinámicamente su contenido a medida que el usuario interactúa con la aplicación. A diferencia de las aplicaciones tradicionales, las SPA no requieren recargar toda la página para mostrar nuevo contenido, lo que resulta en una experiencia más rápida y fluida.

Este enfoque responde a varias necesidades clave del desarrollo moderno:

- 1. Rendimiento y Velocidad:** Al reducir las solicitudes al servidor y minimizar la recarga de páginas completas, las SPA ofrecen tiempos de respuesta más rápidos, mejorando la percepción del rendimiento por parte del usuario.
- 2. Experiencia de Usuario:** Las SPA permiten una navegación más fluida y continua, similar a la de una aplicación de escritorio. Esto mejora significativamente la experiencia del usuario, haciéndola más intuitiva y atractiva.
- 3. Modularidad y Escalabilidad:** Las SPA promueven una arquitectura más modular, donde diferentes partes de la aplicación pueden ser desarrolladas, mantenidas y actualizadas de manera independiente.

En este contexto, Angular se destaca como uno de los frameworks más potentes y populares para desarrollar SPA, proporcionando un conjunto completo de herramientas y prácticas que permiten a los desarrolladores construir aplicaciones escalables, mantenibles y de alto rendimiento. A lo largo de este curso, explicaremos cómo Angular aborda estos desafíos y facilita la creación de aplicaciones modernas y robustas.

Diagrama: Arquitectura de una SPA



En este flujo, el navegador ejecuta la aplicación, solicita datos a una API y actualiza el DOM de forma dinámica sin recargar la página completa. De esta manera se logra una experiencia más fluida y cercana a la de una aplicación de escritorio.

Angular: Un Framework Integral para el Desarrollo de Aplicaciones Web Modernas

Angular es mucho más que una simple biblioteca; es un **framework** de desarrollo de aplicaciones web de código abierto completo y robusto. Es mantenido activamente por Google y una vibrante comunidad de desarrolladores global, lo que asegura su constante evolución y soporte a largo plazo.

Diseñado específicamente para abordar los desafíos de la construcción de aplicaciones web dinámicas y escalables, Angular se ha consolidado como la opción predilecta para desarrollar Single Page Applications (SPA).

El principal atractivo de Angular reside en su enfoque integral y prescriptivo. A diferencia de otras librerías que requieren la integración manual de múltiples herramientas de terceros, Angular proporciona un conjunto coherente y preconfigurado de herramientas, módulos y patrones de diseño (como la arquitectura basada en componentes y la inyección de dependencias) que permiten manejar aplicaciones complejas de manera eficiente, estructurada y mantenible.

Historia y Evolución: De AngularJS a Angular Moderno

La historia de Angular es una de profunda transformación y modernización:
AngularJS

(Angular 1): Los Inicios (2010)

Angular comenzó su andadura en 2010 como una biblioteca llamada **AngularJS** (a menudo referida como Angular 1). Su propósito original era ambicioso: solventar las deficiencias de las técnicas de desarrollo basadas en JavaScript puro de la época, proporcionando un **marco de trabajo más estructurado y declarativo**. Introdujo conceptos revolucionarios como el *Two-Way Data Binding* (enlace bidireccional de datos) y las directivas, que facilitaron la manipulación del DOM y la sincronización de la vista con el modelo. Aunque fue inmensamente popular, AngularJS presentaba limitaciones de rendimiento y una curva de aprendizaje considerable, especialmente en proyectos grandes.

La Gran Reescritura: El Nacimiento de Angular Moderno (Desde Angular 2 en adelante, 2016)

En 2016, Google lanzó una versión completamente nueva y rediseñada desde cero, conocida simplemente como Angular (eliminando el sufijo "JS" para marcar la diferencia con la versión anterior). Esta reescritura no fue una simple actualización, sino un cambio de paradigma total, adoptando tecnologías y prácticas de desarrollo modernas.

La característica fundamental de esta nueva versión es su base en **TypeScript**. TypeScript, desarrollado por Microsoft, es un **superconjunto de JavaScript** que añade características cruciales para el desarrollo a gran escala, siendo la más importante el **tipado estático**. El tipado estático permite a los desarrolladores detectar errores en tiempo de compilación (antes de que la aplicación se ejecute) en lugar de en tiempo de ejecución, lo que mejora drásticamente la calidad, la legibilidad del código y la refactorización.

Esta evolución marcó la transición a una **arquitectura basada en componentes**, que es el estándar actual en el desarrollo frontend. Los componentes son bloques de construcción independientes y reutilizables que encapsulan la lógica, la vista (plantilla HTML) y los estilos (CSS) de una parte específica de la interfaz de usuario. Esta modularidad es clave para la escalabilidad y el mantenimiento de las aplicaciones.

Desde Angular 2 hasta la versión más reciente, el framework ha continuado refinando su rendimiento, mejorando el CLI (Command Line Interface), introduciendo herramientas de renderizado del lado del servidor (Angular Universal) y optimizando el tamaño final de las aplicaciones, consolidándose como una herramienta de grado empresarial.

Características Clave de Angular

Arquitectura Basada en Componentes: Angular organiza la interfaz de usuario en componentes modulares y reutilizables, cada uno con su propia lógica, vista y estilos. Esta arquitectura facilita la gestión del código y el desarrollo de aplicaciones complejas.

- **Inyección de Dependencias:** El sistema de inyección de dependencias de Angular permite gestionar y proporcionar servicios y objetos a los componentes de manera eficiente, mejorando la modularidad y simplificando las pruebas y el mantenimiento del código.
- **Sistema de Ruteo:** Angular incluye un sistema de ruteo que gestiona la navegación entre diferentes vistas sin recargar la página completa. Esto permite una experiencia de usuario fluida y continua, esencial para las SPA.
- **Data Binding Bidireccional:** El enlace bidireccional de datos en Angular sincroniza automáticamente el modelo y la vista, reflejando cualquier cambio en uno de ellos en el otro, lo que reduce la necesidad de código adicional para manejar estos cambios.
- **Directivas:** Las directivas en Angular extienden el HTML con nuevas funcionalidades y comportamientos, permitiendo modificar la estructura del DOM o ajustar el comportamiento y estilo de los elementos.
- **Pipes:** Los pipes transforman datos en las plantillas, permitiendo formatear información como fechas y números directamente en la vista, sin necesidad de lógica adicional en el componente.
- **Formularios:** Angular ofrece dos enfoques para manejar formularios: los formularios basados en plantillas (Template-driven Forms) y los formularios reactivos (Reactive Forms), proporcionando herramientas para la validación y el manejo de datos de entrada del usuario.
- **Angular CLI:** La Angular CLI (Command Line Interface) facilita la creación, desarrollo y mantenimiento de aplicaciones Angular. Permite generar componentes, servicios y módulos, y realizar tareas de construcción y despliegue de manera eficiente.
- **TypeScript:** Uso por defecto
- **Tipado estático:** permite detectar errores durante el desarrollo y no en producción, mejorando la calidad del código.

Actualización 2026: Angular 20 y Angular 21 - La Consolidación de un Framework Moderno

La evolución de Angular ha sido constante, marcando hitos significativos con sus lanzamientos semestrales. Las versiones Angular v20 (mayo 2025) y Angular v21 (noviembre 2025) representan una profunda transformación que orienta el framework hacia una arquitectura más moderna, optimizada en rendimiento y con herramientas de desarrollo de vanguardia, consolidando su posición como una plataforma robusta y ágil.

Angular v20 (Mayo 2025): Eficiencia y Estándares de Rendimiento

Angular v20 se centró en la mejora de la experiencia del desarrollador y el rendimiento de las aplicaciones, introduciendo cambios fundamentales:

- 1. Control Flow Nativo como Estándar:** El nuevo *control flow* declarativo (`@if`, `@for`, `@switch`) que había estado en *developer preview* en versiones anteriores, se convirtió en la sintaxis estándar y predeterminada. Este cambio no solo simplifica la lectura del código en las plantillas de componentes, sino que también ofrece **mejoras sustanciales en el rendimiento** durante el tiempo de ejecución y reduce el tamaño de los *bundles* al eliminar la necesidad de directivas estructurales obsoletas como `*ngIf` o `*ngFor` en nuevas aplicaciones.
- 2. Mejoras en Server-Side Rendering (SSR) con Hidratación Incremental:** Continuó la optimización de SSR. La **hidratación incremental** mejorada permite que las partes interactivas de una aplicación se activen gradualmente en el cliente a medida que se cargan los componentes, en lugar de esperar a que toda la aplicación se cargue y se *hydrate* a la vez. Esto resulta en una **mejora drástica en métricas vitales** como *Largest Contentful Paint (LCP)* y *First Input Delay (FID)*, elementos clave para el SEO y la percepción de velocidad por parte del usuario.
- 3. Detección de Cambios Optimizada y Zoneless por Defecto (Transición):** Se implementaron mejoras en el mecanismo de detección de cambios, facilitando la adopción de una estrategia **zoneless** (sin *NgZone*). Aunque la eliminación completa de *NgZone* como el mecanismo de detección de cambios predeterminado pudo haber sido planificada para versiones ligeramente posteriores, la v20 sentó las bases con herramientas y APIs mejoradas para que los desarrolladores opten por la detección de cambios basada en señales (*signals*) con mayor facilidad, conduciendo a un **menor consumo de memoria** y un **rendimiento más predecible**.

Angular v21 (Noviembre 2025): Testing, Accesibilidad y Modernización de Herramientas

Angular v21 puso el foco en la modernización de la *toolchain* de desarrollo y la expansión de las capacidades del framework, especialmente en áreas críticas como el testing y la accesibilidad:

- 1. Vitest Reemplaza a Karma/Jasmine:** Uno de los cambios más significativos fue la adopción de **Vitest** como el *test runner* predeterminado para nuevas aplicaciones. Esto marca el final de la era de Karma y Jasmine como la configuración por defecto, aprovechando la **velocidad y la eficiencia del ecosistema Vite** para las pruebas unitarias. Vitest ofrece una experiencia de desarrollo de *testing* mucho más rápida, con recarga en caliente (*Hot Module Replacement - HMR*) integrada para las pruebas, mejorando significativamente el *Developer Experience (DX)*.
- 2. Introducción de Angular Aria (Developer Preview):** Pensando en la inclusión y la creación de componentes de alta calidad, v21 introdujo **Angular Aria**. Este conjunto de *primitives* o **componentes headless accesibles** se enfoca en proporcionar la lógica y el estado correctos de WAI-ARIA sin imponer estilos visuales. Los desarrolladores pueden construir *widgets* complejos (como modales, *tooltips*, selectores) con la garantía de que son intrínsecamente accesibles, simplemente añadiendo la capa visual deseada.
- 3. Mejoras Generales en el Rendimiento y la Experiencia de Desarrollo:** Se realizaron optimizaciones continuas en el compilador (Angular CLI y AOT), que resultaron en **tiempos de construcción más rápidos** y *bundles* de producción más pequeños. Estas mejoras de rendimiento no se limitaron a las aplicaciones de usuario final, sino que también mejoraron la velocidad del propio *tooling* del framework.

Conclusión: Un Framework Preparado para el Futuro

Las versiones 20 y 21 de Angular no solo introdujeron nuevas características, sino que consolidaron una estrategia clara: **priorizar el rendimiento, simplificar la sintaxis y modernizar las herramientas internas** para alinear el framework con los estándares más altos del desarrollo web moderno. Estos cambios garantizan que Angular siga siendo una opción potente, eficiente y fácil de mantener para proyectos de cualquier escala, desde aplicaciones empresariales críticas hasta *startups* ágiles.

Herramientas necesarias para el desarrollo con Angular

En esta unidad, profundizaremos en las herramientas y procesos necesarios para trabajar con Angular, comenzando con una introducción a Node.js, Angular CLI y Typescript. También aprenderemos cómo iniciar un proyecto Angular y cómo se estructura este proyecto.

Node.js

Node.js es un entorno de ejecución de JavaScript basado en el motor V8 de Google Chrome. Su principal objetivo es permitir la ejecución de JavaScript del lado del servidor, lo que abre nuevas posibilidades para el desarrollo de aplicaciones web y servidores.

En el contexto de Angular, Node.js es fundamental por las siguientes razones:

- **Gestión de Dependencias:** Node.js utiliza npm (Node Package Manager) para gestionar las dependencias del proyecto. Cuando trabajamos con Angular, npm se encarga de instalar y actualizar Angular y sus bibliotecas, así como otras herramientas necesarias para el desarrollo.
- **Angular CLI:** Angular CLI es una herramienta construida con Node.js. Utiliza Node.js para ejecutar comandos de línea de comandos que generan, construyen y mantienen aplicaciones Angular, facilitando el flujo de trabajo del desarrollo.
- **Automatización de Tareas:** Node.js y npm permiten automatizar tareas de desarrollo como la construcción del proyecto, el linting del código, y la ejecución de pruebas. Esto mejora la eficiencia y asegura que el código siga las mejores prácticas.
- **Servidor de Desarrollo:** Node.js es utilizado por Angular CLI para servir la aplicación en un servidor de desarrollo local, lo que permite ver los cambios en tiempo real mientras desarrollamos.

Documentación

A continuación, te proporcionamos un enlace que engloba toda la información y documentación oficial necesaria para trabajar con Node.js. Puedes acceder a esta valiosa fuente de conocimiento a través del siguiente enlace:

<https://nodejs.org/es/docs>.

El entorno de desarrollo (IDE)

Aunque Angular puede utilizarse con distintos editores, Visual Studio Code se consolidó como una herramienta ampliamente adoptada por su integración con TypeScript. La extensión Angular Language Service brinda autocompletado, verificación de tipos y detección de errores directamente en las plantillas HTML, lo que facilita el desarrollo y reduce el tiempo dedicado a la depuración.

Conocimientos base requeridos

Antes de comenzar con Angular, es importante contar con bases sólidas en:

- **JavaScript moderno:** comprensión de clases, módulos (import/export), funciones de flecha y asincronía.
- **TypeScript:** uso de tipado estático, interfaces y decoradores, que forman parte central de la arquitectura de Angular.
- **HTML5 y CSS3:** estructura semántica y diseño responsivo para construir y estilizar las plantillas.

Errores comunes y diagnóstico

Uno de los inconvenientes más frecuentes es utilizar una versión incompatible de Node.js. Si al instalar Angular CLI aparecen errores de sintaxis o dependencias faltantes, conviene verificar la versión instalada con `node -v`. Angular suele requerir alguna de las últimas versiones estables.

En sistemas Unix o macOS, la instalación global de paquetes (con `-g`) puede generar errores de permisos. En lugar de utilizar `sudo`, resulta más conveniente trabajar con un gestor de versiones como `nvm`, que permite cambiar de versión de Node sin afectar el sistema y evita conflictos relacionados con permisos.

Angular CLI

Angular CLI (Command Line Interface) es una herramienta de línea de comandos desarrollada por el equipo de Angular que facilita la creación, el desarrollo y la administración de proyectos basados en Angular. Angular CLI automatiza muchas de las tareas repetitivas y configuraciones iniciales, lo que agiliza el proceso de desarrollo y garantiza que los proyectos sigan las mejores prácticas recomendadas por el equipo de Angular.

Permite realizar, entre muchas más, las siguientes acciones:

- **Generación de Código:** Angular CLI permite crear componentes, servicios, módulos y otros artefactos de la aplicación con comandos simples
- **Configuración del Proyecto:** CLI gestiona automáticamente la configuración de tu proyecto, incluyendo el archivo angular.json que define las opciones de construcción y configuración de la aplicación.
- **Gestión de Dependencias:** Angular CLI utiliza npm para gestionar las dependencias del proyecto. El archivo package.json lista todas las dependencias necesarias, y puedes instalar o actualizar estas dependencias utilizando comandos npm. Angular CLI automáticamente actualiza package.json cuando instalas nuevas dependencias o creas un nuevo proyecto
- **Construcción y Despliegue:** Con comandos como ng build, Angular CLI construye y optimiza la aplicación para producción, manejando la minificación y la optimización de los recursos.
- **Servidor de Desarrollo:** El comando ng serve inicia un servidor de desarrollo que permite ejecutar y probar la aplicación localmente, con recarga automática de cambios durante el desarrollo.

Comando	Acción
ng new	Crea un nuevo proyecto con la estructura recomendada.
ng serve	Inicia un servidor de desarrollo con recarga automática.
ng generate	Crea componentes, servicios o pipes siguiendo las convenciones.
ng build	Compila la aplicación para producción, optimizando el tamaño de los archivos.

Documentación

<https://angular.dev/tools/cli>

Configuración Angular CLI

Antes de comenzar a ejecutar los comandos del **Angular CLI** es necesario contar **NodeJS** y **NPM** en el sistema operativo. Siempre se debe descargar desde la página oficial para evitar descargar software malicioso.

La página oficial de node es la siguiente: <https://nodejs.org/> aquí podrá encontrar varias versiones y para varios sistemas operativos, a la hora de elegir una versión se recomienda siempre trabajar con las versiones LTS las cuales nos dan soporte a largo plazo a diferencia de las STS.

Comandos

Todos los comandos serán ejecutados en una terminal (cmd, bash, etc), incluso es posible utilizar la terminal integrada de VSCode. Lo primero que es necesario realizar es instalar Angular CLI lo que nos va a permitir utilizar comandos para la creación de los distintos elementos que proporciona angular. Para ello recurriremos al administrador de paquetes de node (NPM) ejecutando el siguiente comando:

```
npm install -g @angular/cli
```

Descomposición de comando:

- **npm**: es la librería que se está llamando, como se nombró anteriormente esta hace referencia a Node Package Manager (Administrador de Paquetes de Node).
- **install**: hace referencia a la operación que se realizará, en este caso instalar.
- **-g**: es un parámetro opcional que indica que dicho paquete va a ser instalado de manera global en el equipo para que esté disponible en cualquier lugar que se necesite, por ejemplo, en caso de crear un nuevo proyecto al hacerlo de esta manera no será necesario instalarlo nuevamente.
- **@angular/cli**: es el nombre del paquete que se desea instalar, en este caso es el Command Line Interface de Angular.

Errores comunes y diagnóstico

- **Errores de tipeo en los comandos**: es habitual escribir mal algún comando, por ejemplo ng sevre en lugar de ng serve. El CLI suele mostrar sugerencias para corregirlo.
- **Restricciones en PowerShell (Windows)**: en algunos casos la ejecución de scripts está bloqueada. Puede ser necesario ejecutar:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope  
CurrentUser
```

Esto habilita la ejecución de scripts firmados localmente y permite que el CLI funcione correctamente.

Actualización 2026: ng update y migración automática

El comando `ng update` permite actualizar Angular a la última versión de forma guiada, aplicando migraciones automáticas (schematics) que actualizan código obsoleto. Por ejemplo, al migrar a Angular 21 se puede convertir tests de Jasmine a Vitest con: `ng g @schematics/angular:refactor-jasmine-vitest`.

Vitest es el test runner predeterminado en Angular 21: es más rápido que Karma, compatible con ESM nativo y se configura automáticamente al crear un proyecto nuevo.

Creando nuestro primer proyecto de Angular

Una vez que se tiene el Angular CLI ya es posible crear un nuevo proyecto en Angular. Para ello hay que ejecutar el siguiente comando:

```
ng new nombre-del-proyecto
```

Dicho comando realizará algunas preguntas sobre el routing (por ahora podemos poner que no) y los estilos que desea aplicar (por el momento podemos utilizar el ofrecido por defecto "css").

Descomposición de comando:

- **ng**: hace referencia a Angular.
- **new**: operación que se va a realizar, en este caso crear un nuevo proyecto.
- **nombre-del-proyecto**: este es el nombre que le desea poner al proyecto. En caso de tener separaciones en el nombre del proyecto es necesario indicarlo con un guión medio.

¿Habilitar Server-Side Rendering (SSR)?

Puede ser útil si se necesita optimizar el SEO o mejorar el rendimiento inicial en aplicaciones públicas.

¿Qué formato de estilos utilizar?

CSS es la opción estándar. SCSS ofrece más funcionalidades, como variables y anidamiento.

¿Incluir enrutamiento?

Es recomendable activarlo si la aplicación tendrá múltiples vistas o secciones.

Una vez finalizado el proceso, se obtiene una estructura base lista para comenzar a desarrollar, con configuración de TypeScript, scripts de ejecución y un servidor de desarrollo preparado para trabajar de forma ágil.

Una vez finalizado el proceso de creación del proyecto lo siguiente es correr el mismo. Para ello será necesario utilizar un comando para iniciar el servidor de desarrollo integrado. Este servidor se encarga de compilar la aplicación Angular en el entorno local, facilitando la visualización y probar la aplicación mientras se desarrolla. El comando que se usará es:

ng serve

Luego de ejecutar este comando se mostrará un mensaje indicando el servidor de desarrollo que se creó. Internamente este comando compila el código TypeScript de la aplicación en código JavaScript creando un conjunto de archivos que el navegador pueda entender. Además, monitorea los cambios en los archivos de la aplicación y recarga automáticamente la página del navegador cada vez que detecta un cambio en el código fuente. Esto facilita el proceso de desarrollo, ya que permite ver inmediatamente los resultados de los cambios realizados en la aplicación.

Primer proyecto: comandos y estructura

Crear el primer proyecto marca el inicio formal de la aplicación. El comando **ng new** no solo genera carpetas, sino que configura un entorno completo de compilación, dependencias y buenas prácticas por defecto.

Estructura del proyecto

Cada una de las carpetas generadas en un proyecto de Angular tienen un propósito específico. Se presenta una estructura típica de un proyecto:

```

nombre-del-proyecto/
├── node_modules/           # Dependencias instaladas por npm (no se versiona)
├── src/                    # Código fuente de la aplicación
│   ├── app/               # Núcleo de la aplicación Angular
│   │   ├── components/    # Componentes reutilizables (UI compartida)
│   │   │   ├── header/
│   │   │   │   ├── header.component.ts
│   │   │   │   ├── header.component.html
│   │   │   │   ├── header.component.scss
│   │   │   │   └── header.component.spec.ts
│   │   │   └── footer/
│   │   │       └── ...
│   │   ├── pages/        # Componentes de página (una por ruta)
│   │   │   ├── home/
│   │   │   │   ├── home.component.ts
│   │   │   │   ├── home.component.html
│   │   │   │   └── home.component.spec.ts
│   │   │   └── about/
│   │   │       └── ...
│   │   ├── services/      # Servicios e inyección de dependencias
│   │   │   ├── auth.service.ts
│   │   │   └── api.service.ts
│   │   ├── models/        # Interfaces y tipos TypeScript
│   │   │   └── user.model.ts
│   │   ├── guards/        # Guards de navegación (canActivate, etc.)
│   │   │   └── auth.guard.ts
│   │   ├── interceptors/  # Interceptores HTTP (tokens, errores)
│   │   │   └── auth.interceptor.ts
│   │   ├── pipes/         # Pipes personalizados
│   │   │   └── truncate.pipe.ts
│   │   ├── directives/    # Directivas personalizadas
│   │   │   └── highlight.directive.ts
│   │   ├── app.component.ts      # Componente raíz de la aplicación
│   │   ├── app.component.html    # Template del componente raíz
│   │   ├── app.component.scss    # Estilos del componente raíz
│   │   ├── app.component.spec.ts # Tests del componente raíz
│   │   ├── app.config.ts         # Configuración de la app (providers)
│   │   └── app.routes.ts        # Definición de rutas de la aplicación
│   ├── assets/               # Recursos estáticos
│   │   ├── images/          # Imágenes (.png, .svg, .jpg)
│   │   ├── fonts/           # Fuentes personalizadas
│   │   └── i18n/            # Archivos de internacionalización
│   ├── environments/        # Configuración por entorno
│   │   ├── environment.ts    # Variables de entorno (desarrollo)
│   │   └── environment.prod.ts # Variables de entorno (producción)
│   ├── index.html           # Página HTML principal (punto de entrada)
│   ├── main.ts              # Bootstrap de la aplicación Angular
│   └── styles.scss           # Estilos globales (CSS/SCSS)
├── angular.json             # Configuración del workspace Angular (build, serve, test)
├── package.json             # Dependencias, scripts npm y metadatos del proyecto
├── package-lock.json        # Versiones exactas de dependencias (lock file)
├── tsconfig.json            # Configuración base del compilador TypeScript
├── tsconfig.app.json        # Configuración TS específica para la app
├── tsconfig.spec.json       # Configuración TS específica para tests
├── .editorconfig            # Reglas de formato del editor (tabs, espacios)
├── .gitignore               # Archivos excluidos del control de versiones
└── README.md               # Documentación del proyecto
    
```

Dicha estructura puede variar dependiendo los equipos y definiciones que puedan tener los distintos equipos aunque los archivos creados por angular al crear el proyecto es recomendable no modificarlos.

La carpeta **node_modules**, que contiene todas las dependencias, no debe ser incluida en el repositorio. Por lo tanto, cuando se trabaja en proyectos que ya han sido creados, es necesario regenerar esta carpeta en el entorno local. Para lograrlo, simplemente ejecuta el siguiente comando de instalación de paquetes en la misma ubicación donde se encuentra el archivo **package.json**.

```
npm install
```

Este comando restablecerá las dependencias necesarias según las especificaciones del archivo **package.json**.

En Angular moderno la estructura se simplifica y se orienta a componentes autónomos (standalone). En lugar de depender de módulos para configurar la aplicación, la configuración principal se concentra en archivos de arranque y configuración global.

Árbol de archivos principal

- **src/app/**: el núcleo de la aplicación. Acá viven los componentes y la lógica principal.
- **app.config.ts**: centraliza proveedores globales (por ejemplo, cliente HTTP y routing) sin recurrir a módulos.
- **main.ts**: arranca la aplicación (bootstrap) y conecta la configuración global con el componente raíz.
- **angular.json**: define cómo se construye el proyecto (build, assets, estilos, configuraciones por entorno, etc.).
- **package.json**: lista librerías instaladas y scripts como start, build y test.

Comandos de ejecución

Una vez creado el proyecto, entramos a la carpeta y levantamos el servidor de desarrollo:

```
cd mi-primer-app ng serve -o
```

El parámetro -o abre automáticamente el navegador en <http://localhost:4200>.

Errores comunes y diagnóstico

Puerto en uso: si otro proceso está usando el puerto 4200, Angular no podrá iniciar. Podemos cambiarlo así:

```
ng serve --port 4201
```

Dependencias faltantes: si se borró la carpeta node_modules, la app no va a levantar. Se recupera ejecutando:

```
npm install
```

Componentes y Standalone Components

¿Qué son los Componentes en Angular?

En Angular, un componente es una unidad fundamental de la interfaz de usuario (UI). Cada componente es una pieza de la interfaz que encapsula su lógica, datos y presentación. Los componentes permiten construir aplicaciones modulares y reutilizables, dividiendo la interfaz en partes manejables y organizadas.

¿Para qué sirven los componentes?

- **Encapsulamiento:** Los componentes encapsulan la lógica de la interfaz, mejorando la organización del código.
- **Reusabilidad:** Pueden reutilizarse en diferentes partes de la aplicación, evitando duplicación.
- **Mantenimiento:** Facilitan actualizaciones y cambios en partes específicas sin afectar al resto.

Creación de un Componente

Componentes Tradicionales (con Módulos)

Para crear un componente tradicional (antes de Angular 17), se usa el CLI:

```
ng generate component nombre-del-componente
```

Componentes Autónomos (Standalone Components)

A partir de Angular 17/18, se introdujeron los Standalone Components, que no requieren módulos. Para crearlos, usa:

```
ng generate component nombre-del-componente --standalone
```

```
ng g c nombre-del-componente --standalone
```

Descomposición del Comando

- **ng**: Referencia a Angular.
- **generate (o g)**: Operación para generar un elemento.
- **component**: Tipo de elemento a crear.
- **nombre-del-componente**: Nombre del componente (usando guiones para separar palabras).
- **--standalone**: Opción para crear un componente autónomo (Angular 17+).

Estructura de un Componente

Componente Tradicional

El CLI genera:

- **nombre-del-componente.component.ts**: Lógica en TypeScript.
- **nombre-del-componente.component.html**: Plantilla HTML.
- **nombre-del-componente.component.css**: Estilos encapsulados.
- **nombre-del-componente.component.spec.ts**: Pruebas unitarias.

Ejemplo del archivo **.ts** tradicional:

```
import { Component } from '@angular/core';

no usages
@Component({
  selector: 'app-mi-componente',
  templateUrl: './mi-componente.component.html',
  styleUrls: ['./mi-componente.component.css']
})
export class MiComponente {
  // Lógica del componente
}
```

Componente Autónomo (Standalone)

En Angular 17+, el componente incluye **standalone: true** y gestiona sus propias dependencias:

```
import { Component } from '@angular/core';

no usages
@Component({
  selector: 'app-mi-componente',
  templateUrl: './mi-componente.component.html',
  styleUrls: ['./mi-componente.component.css'],
  standalone: true, // Indica que es autónomo
  imports: [/* Componentes o directivas usadas en la plantilla */]
})
export class MiComponente {
  // Lógica del componente
}
```

Uso de un Componente

Componentes Tradicionales

Deben declararse en un módulo. Ejemplo en app.module.ts:

```
import { MiComponente } from './mi-componente/mi-componente.component';

no usages
@NgModule({
  declarations: [MiComponente],
  // ...
})
export class AppModule { }
```

Luego, se usa en HTML con su selector:

```
<app-mi-componente></app-mi-componente>
```

Componentes Autónomos (Standalone)

No requieren módulos. Se importan directamente donde se necesitan:

```
import { Component } from '@angular/core';
import { MiComponente } from './mi-componente/mi-componente.component';

no usages
@Component({
  selector: 'app-otro-componente',
  templateUrl: './otro-componente.component.html',
  styleUrls: ['./otro-componente.component.css'],
  standalone: true,
  imports: [MiComponente] // Importa el componente autónomo aquí
})
export class OtroComponente {
  // Lógica del componente
}
```

Y en HTML:

```
<app-mi-componente></app-mi-componente>
```

Ventajas de los Standalone Components (Angular 17+)

- **Sin módulos:** Elimina la necesidad de declarar componentes en módulos.
- **Código más limpio:** Reduce archivos de configuración y boilerplate.
- **Lazy Loading simplificado:** Carga componentes directamente en rutas sin módulos.
- **Reusabilidad mejorada:** Los componentes son autocontenidos y fáciles de compartir.

Configuración de Rutas con Standalone Components

En Angular 17+, puedes definir rutas usando componentes autónomos:

```
import { Routes } from '@angular/router';
import { MiComponente } from './mi-componente/mi-componente.component';

no usages
export const routes: Routes = [
  {
    path: 'ruta',
    component: MiComponente, // Componente autónomo
  }
];
```

Inyección de Dependencias en Standalone Components

Funciona igual que en componentes tradicionales. Ejemplo con un servicio:

```
import { Component } from '@angular/core';
import { MiServicio } from './mi-servicio.service';

no usages
@Component({
  selector: 'app-mi-componente',
  templateUrl: './mi-componente.component.html',
  styleUrls: ['./mi-componente.component.css'],
  standalone: true
})
export class MiComponente {
  no usages
  constructor(private miServicio: MiServicio) {
    // Usa el servicio aqui
  }
}
```

¿Cuándo usar Standalone Components?

- **Aplicaciones nuevas:** Angular 17+ recomienda usar componentes autónomos como estándar.
- **Refactorización:** Migrar gradualmente componentes antiguos a autónomos.
- **Librerías:** Para crear componentes reutilizables sin depender de módulos.

Conclusión

Los **Standalone Components** son una evolución clave en Angular (versiones 17/18) que simplifican la arquitectura de las aplicaciones, eliminando la necesidad de módulos en la mayoría de los casos. Esta característica permite un desarrollo más ágil, código más limpio y una mejor escalabilidad. Se recomienda adoptarlos en proyectos nuevos y migrar progresivamente los existentes.

Errores comunes y diagnóstico

Olvidar standalone: true: en tutoriales antiguos esto no aparece. Si intentamos usar un componente sin declararlo en un módulo y tampoco es standalone, Angular mostrará un error de compilación.

Falta de importaciones: si dentro del template usamos directivas, pipes u otros componentes, deben declararse en **imports: [...]** del decorador **@Component**. Esto aplica tanto a features nuevas como a dependencias comunes (por ejemplo, directivas de Angular o componentes propios).

Ciclo de Vida de Componentes

Los componentes y directivas en Angular no son entidades estáticas; atraviesan un **ciclo de vida** riguroso y bien orquestado. Este ciclo se compone de una secuencia de etapas o eventos clave, que inician con la creación de la instancia del componente o directiva y culminan con su eventual eliminación del DOM y la memoria.

Este mecanismo de ciclo de vida es crucial porque proporciona a los desarrolladores puntos de intervención específicos, conocidos como "hooks" (ganchos), para ejecutar lógica personalizada. Entender y utilizar estos hooks permite:

1. **Inicialización:** Configurar el estado inicial, cargar datos y establecer suscripciones.
2. **Reacción a Cambios:** Responder eficientemente a modificaciones en las propiedades de entrada (**@Input**) o en el contenido proyectado.
3. **Control del Renderizado:** Manipular el DOM o la vista después de su inicialización o actualización.
4. **Limpieza de Recursos:** Liberar memoria y prevenir pérdidas (memory leaks) antes de la destrucción.

Se pueden implementar estos "hooks" de dos maneras:

1. Implementando explícitamente la interfaz correspondiente (ej. **OnInit**, **OnChanges**). Esta es la práctica recomendada para garantizar la correcta tipificación.
2. Simplemente definiendo el método asociado (ej. **ngOnInit()**) en la clase del componente/directiva.

Eventos Clave del Ciclo de Vida (Hooks)

Los hooks se disparan en una secuencia predecible y estricta, permitiendo un control granular sobre el comportamiento de la aplicación. 1. Inicialización y Reacción a Entradas

Hook	Interfaz	Descripción y Uso Típico	Secuencia de Ejecución
ngOnChanges	OnChanges	Reacciona a cambios en los datos de entrada. Se dispara cada vez que Angular detecta un cambio en las propiedades de entrada decoradas con <code>@Input()</code> . Recibe un objeto <code>SimpleChanges</code> que contiene el valor actual, el valor anterior y si es el primer cambio.	Se ejecuta antes de <code>ngOnInit</code> y múltiples veces después si las propiedades de entrada cambian.
ngOnInit	OnInit	Ideal para inicialización única. Este es el lugar estándar para inicializar la lógica del componente, realizar llamadas a servicios (fetching de datos) o configurar propiedades una vez que Angular ha terminado de inicializar las propiedades de entrada.	Se dispara una sola vez, inmediatamente después del primer <code>ngOnChanges</code> (o directamente si no hay inputs), y antes de que la vista se muestre.
ngDoCheck	DoCheck	Detección de cambios personalizada. Se utiliza para implementar lógica de detección de cambios manual o más granular, especialmente cuando el mecanismo predeterminado de <code>ngOnChanges</code> es insuficiente (ej. para monitorear objetos mutables que no disparan <code>ngOnChanges</code>).	Se dispara en cada ciclo de detección de cambios de la aplicación, incluso si no hay cambios en las propiedades de entrada. Es un hook intensivo y debe usarse con precaución.

2. Contenido Proyectado (Content Projection)

Estos hooks se aplican a elementos que son "proyectados" dentro del componente anfitrión mediante `<ng-content>`.

Hook	Interfaz	Descripción y Uso Típico	Secuencia de Ejecución
ngAfterContentInit	AfterContentInit	Inicialización del contenido proyectado. Útil para trabajar con las referencias a elementos o componentes obtenidos del contenido proyectado (usando <code>@ContentChild()</code> o <code>@ContentChildren()</code>) después de que hayan sido completamente inicializados.	Se ejecuta una sola vez , después del primer <code>ngDoCheck</code> y después de que Angular haya inicializado todo el contenido proyectado.
ngAfterContentChecked	AfterContentChecked	Verificación del contenido proyectado. Permite reaccionar a cambios en el contenido que ha sido proyectado dentro del componente.	Se ejecuta después de cada <code>ngDoCheck</code> y después de que Angular haya verificado el contenido proyectado.

3. Vista del Componente (Component View)

Estos hooks se refieren a los elementos definidos en la plantilla (`templateUrl` o `template`) del propio componente.

Hook	Interfaz	Descripción y Uso Típico	Secuencia de Ejecución
<code>ngAfterViewInit</code>	<code>AfterViewInit</code>	Inicialización de la vista propia. Es el momento ideal para interactuar directamente con los elementos de la vista del componente (obtenidos con <code>@ViewChild()</code> o <code>@ViewChildren()</code>) o para inicializar librerías de terceros basadas en el DOM.	Se ejecuta una sola vez , después de <code>ngAfterContentChecked</code> y después de que la vista del componente y las vistas de sus hijos se hayan inicializado.
<code>ngAfterViewChecked</code>	<code>AfterViewChecked</code>	Verificación de la vista propia. Se usa para realizar acciones o reaccionar a cambios que ocurrieron en la vista del componente después de que Angular la ha verificado.	Se ejecuta después de cada <code>ngAfterContentChecked</code> y después de que Angular ha verificado la vista del componente.

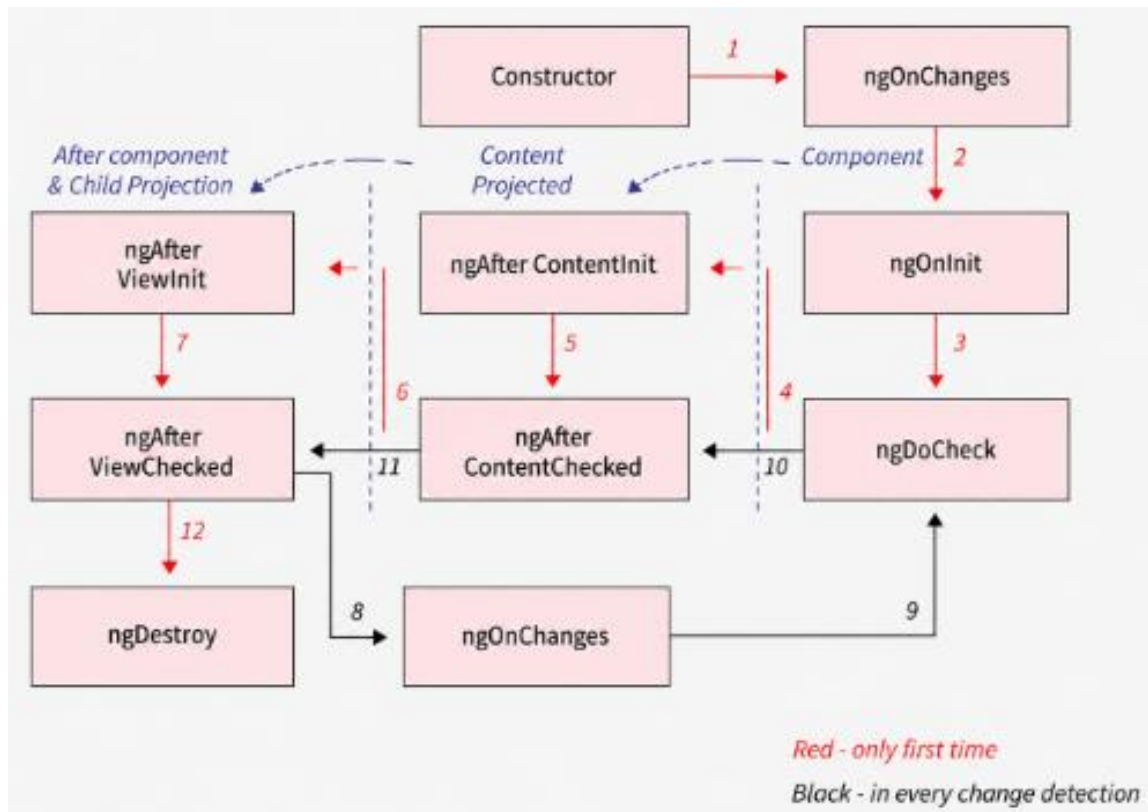
4. Destrucción

Hook	Interfaz	Descripción y Uso Típico	Secuencia de Ejecución
<code>ngOnDestroy</code>	<code>OnDestroy</code>	Limpieza de recursos. Es el hook esencial para liberar recursos. Debe utilizarse para desuscribirse de observables (para evitar pérdidas de memoria), cancelar <i>timers</i> o <i>intervals</i> , y desvincular <i>event handlers</i> que no serán limpiados automáticamente.	Se ejecuta inmediatamente antes de la destrucción del componente o directiva.

Flujo de Ejecución Típico

El ciclo sigue un patrón estricto cuando un componente es inicializado y luego actualizado:

Fase de Inicialización:	Fase de Actualización (Cambios Detectados):	Fase de Destrucción:
- Constructor <code>ngOnChanges</code> (si hay <code>@Input()</code>) <code>ngOnInit</code> <code>ngDoCheck</code> <code>ngAfterContentInit</code> <code>ngAfterContentChecked</code> <code>ngAfterViewInit</code> <code>ngAfterViewChecked</code>	<code>ngOnChanges</code> (si los inputs cambian) <code>ngDoCheck</code> <code>ngAfterContentChecked</code> <code>ngAfterViewChecked</code>	<code>ngOnDestroy</code>



Binding

One way binding

Se refiere a la forma en que los datos fluyen desde el componente de Angular hacia la vista (la plantilla). Permite que los valores de las propiedades del componente se reflejen en la vista, pero no permite que los cambios en la vista afecten directamente las propiedades del componente. Esto garantiza una dirección única de flujo de datos, lo que simplifica el seguimiento de cómo los datos se actualizan y evita posibles problemas de sincronización.

Hay dos tipos posibles:

- 1. Interpolación:** la interpolación se utiliza para mostrar valores de propiedades del componente en la vista. Se realiza utilizando dobles llaves. Por ejemplo:

```
<p>{{ mensaje }}</p>
```

Donde mensaje es una propiedad del componente (clase typescript), por ende dicho valor será mostrado dentro de las etiquetas de párrafo:

```
export class SaludoComponent {
  mensaje: string = '¡Hola, mundo!';
}
```

2. **Enlace de propiedades:** conocido como “**property binding**”, permite enlazar una propiedad del DOM a una propiedad del componente. Esto se realiza utilizando corchetes []. Por ejemplo:

Componente:

```
export class EjemploComponent {  
  imageUrl: string = 'https://example.com/imagen.jpg';  
}
```

Template:

```
<img [src]="imageUrl" alt="Imagen de ejemplo">
```

En este caso el nombre es una propiedad del componente, por lo que el valor de la imagen será establecido con el valor que tenga la propiedad imageUrl del componente. En el caso del enlace unidireccional, los cambios en el componente se reflejarán en la vista automáticamente. Sin embargo, si los valores en la vista cambian esos cambios no se reflejarán en el componente de forma directa.

Two way binding

Es una característica que permite mantener sincronizados automáticamente los valores entre una propiedad del componente y un elemento en la vista, como un campo de entrada. Esto significa que los cambios realizados en la vista se reflejarán en el componente y viceversa, lo que simplifica la gestión de los datos entre ambas direcciones.

Para ello debemos agregar un evento que modifique el valor de la propiedad que se haya asociado a un elemento de la vista. A continuación, se plantea un ejemplo:

El evento (**input**) se activa cada vez que el contenido del campo de entrada cambia. El método **onNombreChange()** actualiza la propiedad nombre del componente con el valor del campo de entrada.

Existen otras alternativas más sencillas como el uso de la directiva **ngModel** que **será visto en otra unidad**.

Resumen comparativo de Data Binding en Angular

Tipo de Binding	Sintaxis	Dirección de Flujo de Datos	Uso Común
Interpolación	<code>{{ ... }}</code>	Componente --> Vista	Mostrar texto dinámico o valores de propiedades.
Property Binding	<code>[propiedad]="expresión"</code>	Componente --> Vista	Cambiar atributos de elementos HTML (ej. <code>src</code> , <code>disabled</code> , <code>value</code>).
Event Binding	<code>(evento)="método()"</code>	Vista --> Componente	Capturar eventos del usuario (ej. <code>click</code> , <code>keyup</code> , <code>submit</code>).
Two-Way Data Binding	<code>[(ngModel)]="propiedad"</code>	Ambos sentidos	Sincronización automática en formularios y elementos de entrada de datos (<code><input></code> , <code><textarea></code> , etc.).

Errores comunes y diagnóstico

Sintaxis incorrecta en two-way binding: escribir `[[ngModel]]` en lugar de `[(ngModel)]` genera un error. Angular suele indicar claramente el problema en la consola.

Propiedades inexistentes: si se intenta vincular una variable que no está declarada en la clase TypeScript, aparecerá un error como "Property does not exist on type...".

Directivas

Son una parte fundamental que te permiten manipular el DOM, aplicar lógica y cambiar la apariencia de los elementos en la vista. Las directivas son instrucciones que se agregan a los elementos HTML para modificar su comportamiento o apariencia. Hay tres tipos principales de directivas en Angular:

Directivas estructurales

Modifican la estructura del DOM, agregando, eliminando o reemplazando elementos HTML en función de condiciones. Las directivas estructurales más comunes son **ngIf**, **ngFor** y **ngSwitch**.

- **ngIf**: agrega o elimina elementos del DOM basados en una condición.

```
<div *ngIf="condicion">Este elemento se mostrará si condicion es verdadero</div>
```

- **ngFor**: itera sobre una colección y genera elementos en función de los elementos de la colección.

```
import { Component } from '@angular/core';

no usages
@Component({
  selector: 'app-root',
  template: `
    <ul>
      <li *ngFor="let item of items">{{ item }}</li>
    </ul>
  `,
})
export class AppComponent {
  items: string[] = [
    'uno', 'dos', 'tres'
  ];
};
```

- **ngSwitch**: muestra un bloque de elementos HTML basado en múltiples condiciones. Misma función que el switch de los lenguajes de programación más utilizados.

```
import { Component } from '@angular/core';

no usages
@Component({
  selector: 'app-root',
  template: `
    <div [ngSwitch]="opcion">
      <p *ngSwitchCase="'opcion1'">Opción 1 seleccionada</p>
      <p *ngSwitchCase="'opcion2'">Opción 2 seleccionada</p>
      <p *ngSwitchDefault>Opción por defecto</p>
    </div>
  `,
})
export class AppComponent {
  opcion: string = 'opcion1';
};
```

Directivas de atributos

Se aplican como atributos en elementos HTML para proporcionar comportamientos específicos. Las directivas de atributo más conocidas son **ngClass**, **ngStyle**.

- **ngClass**: Permite cambiar las clases CSS de un elemento en función de condiciones.

```
import { Component } from '@angular/core';

no usages
@Component({
  selector: 'app-root',
  template: `
    <div [ngClass]="{'clase-activa': estaActiva, 'clase-inactiva': !estaActiva}">
      Este elemento tendrá una clase diferente basada en estaActiva
    </div> `,
})
export class AppComponent {
  estaActiva: boolean = true;
}
```

- **ngStyle**: Permite aplicar estilos CSS a un elemento en función de condiciones.

```
import { Component } from '@angular/core';

no usages
@Component({
  selector: 'app-root',
  template: `
    <div [ngStyle]="{'color': estaActiva ? 'green' : 'red', 'font-weight': estaActiva ? 'bold' : 'normal'}">
      Este elemento tendrá una clase diferente basada en estaActiva
    </div> `,
})
export class AppComponent {
  estaActiva: boolean = true;
}
```

Estos ejemplos demuestran cómo puedes usar directivas de atributos para cambiar la apariencia, el comportamiento y las interacciones de los elementos en función de condiciones dinámicas.

Directivas personalizadas

Son directivas creadas por los desarrolladores para aplicar comportamientos y lógica personalizados a los elementos del DOM. Las directivas personalizadas pueden ser directivas estructurales o de atributo.

Para crear una directiva personalizada en Angular, puedes utilizar el comando **ng generate directive <nombre-de-la-directiva>**, también es posible creando los archivos manualmente.

Código directiva:

```
import { Directive, ElementRef, Input, OnInit } from '@angular/core';

no usages
@Component({
  selector: 'appColorFondo'
})
export class ColorFondoDirective implements OnInit {
  @Input() appColorFondo: string;

  no usages
  constructor(private el: ElementRef) {}

  no usages
  ngOnInit(): void {
    this.el.nativeElement.style.backgroundColor = this.appColorFondo || 'green';
  }
}
```

En este caso se muestra un ejemplo de una directiva que modifica el color de fondo del elemento donde se la utilice.

Utilización directiva:

```
<div appColorFondo="red">Este elemento tendrá color rojo</div>
<div appColorFondo>
  Este elemento tendrá color de fondo verde (por defecto) ya que en caso de que
  la primera expresión sea falsa (en este caso lo será cuando sea nula o indefinida)
  tomará la segunda en su lugar.
</div>
```

El valor del color de fondo es tomado desde el valor proporcionado en el atributo declarado con el mismo nombre que la directiva. Es posible declarar otros atributos y los mismos van a ser llamados como una propiedad de entrada vista anteriormente en la comunicación de componentes.

Otras directivas

ng-content

Es una directiva en Angular que se utiliza para transmitir contenido desde un componente padre a un componente hijo. Permite que el componente padre inserte contenido dentro del componente hijo en la ubicación marcada por `<ng-content>`. Esto es especialmente útil cuando deseas crear componentes reutilizables y flexibles que puedan adaptarse al contenido específico proporcionado por el componente padre.

La directiva `<ng-content>` se coloca en la plantilla del componente hijo donde se desea que se inserte el contenido transmitido. El contenido transmitido puede ser cualquier cosa, desde texto simple hasta componentes más complejos.

```
<!-- card.component.html -->
<div class="card">
  <ng-content></ng-content>
</div>

<!-- parent.component.html -->
<app-card>
  <h2>Título</h2>
  <p>Contenido del parrafo.</p>
</app-card>
```

El contenido proporcionado entre las etiquetas `<app-card></app-card>` en **ParentComponent** se insertará en el lugar donde se encuentra `<ng-content>` en la plantilla de **CardComponent**.

ng-template

Es una estructura en Angular que te permite definir contenido que no se mostrará en la página de manera predeterminada, pero que puede ser referenciado y renderizado cuando sea necesario. Se utiliza en conjunto con otras directivas, como `*ngIf`, `*ngFor` y `*ngSwitch`, para controlar cuándo y dónde se muestra el contenido.

```
<!-- card.component.html -->
<ng-template #bloqueElse>
  <ng-content></ng-content>
</ng-template>

<!-- parent.component.html -->
<div *ngIf="mostrarContenido; else bloqueElse">Contenido condicional</div>
```

En este caso **mostrarContenido** es una variable en el componente typescript la cual tiene un valor booleano. En caso de ser positiva va a mostrar el contenido del párrafo. De lo contrario el sistema mostrará lo que se haya definido en el template que está renombrado como **bloqueElse**.

ng-container

Es una estructura en Angular que actúa como un contenedor temporal y no se renderiza en el DOM final. Se utiliza para agrupar elementos y aplicar lógica sin agregar elementos adicionales al DOM. Esta estructura es especialmente útil cuando deseas manipular o aplicar directivas a múltiples elementos sin afectar la estructura del DOM resultante.

- **Agrupacion de elementos:** para agrupar elementos sin añadir un elemento adicional al DOM. Esto es útil cuando necesitas aplicar directivas o lógica a varios elementos sin agregar un contenedor adicional en el DOM.

```
@for (item of items) {  
  <ng-container>  
    <div>{{ item }}</div>  
  </ng-container>  
}
```

- **En directivas estructurales:** permite aplicar estas directivas a un grupo de elementos sin agregar un elemento extra al DOM.

```
@if (mostrarElemento) {  
  <ng-container>  
    <div>Este elemento se mostrará si mostrarElemento es verdadero</div>  
  </ng-container>  
}
```

- **Evitar Elementos Adicionales en el DOM:** una ventaja es que no agrega elementos adicionales al DOM final. Esto puede ser útil cuando deseas aplicar lógica o estructuras a tu plantilla sin afectar la estructura de salida.

Nuevas Directivas Estructurales en Angular 18: @if, @else, @for y @switch

Angular 18 marca un hito en la simplificación y modernización del desarrollo *front-end* al introducir un conjunto de directivas estructurales nativas y muy esperadas: `@if`, `@else`, `@for` y `@switch`. Estas directivas han sido diseñadas con el objetivo primordial de optimizar la legibilidad, la claridad y el rendimiento del código dentro de los *templates* de Angular. Representan una evolución significativa y sirven como un reemplazo directo y altamente recomendado para las directivas estructurales tradicionales basadas en prefijos como `*ngIf`, `*ngFor` y `*ngSwitch`, que han sido pilares del *framework* desde sus inicios. Este cambio no solo alinea a Angular con tendencias de sintaxis más declarativas, sino que también ofrece beneficios tangibles en el proceso de *build* y en el rendimiento de tiempo de ejecución. A continuación, se detalla en profundidad la funcionalidad y las ventajas de cada una de estas nuevas herramientas.

@if y @else: Renderizado Condicional Mejorado y Simplificado

La directiva @if

Es la nueva herramienta estándar para gestionar el renderizado condicional de elementos en el DOM. Su propósito es controlar si un bloque de contenido específico debe incluirse o excluirse del *template* basándose en la evaluación de una expresión booleana.

Sintaxis y Claridad:

La sintaxis del `@if` es intuitiva y se asemeja mucho a las estructuras condicionales que se encuentran en lenguajes de programación tradicionales, lo que reduce la curva de aprendizaje y mejora la claridad del código inmediatamente:

```
@if (usuarioLogueado) {  
  <p>Bienvenido, {{ nombreUsuario }}</p>  
}
```

Uso de @else:

La directiva `@else` potencia el `@if` al proporcionar un mecanismo elegante para definir un bloque de contenido alternativo. Este contenido se renderizará de forma exclusiva cuando la expresión booleana asociada al `@if` principal se evalúe como falsa (`false`).

La principal ventaja de este nuevo enfoque es que integra la lógica condicional completa (el caso positivo y el alternativo) en un único bloque contiguo dentro del *template*.

```
@if (productosEnStock > 0) {  
  <ul>  
    <li>Producto A</li>  
    <li>Producto B</li>  
  </ul>  
} @else {  
  <p>Lo sentimos, no hay productos disponibles en este momento.</p>  
}
```

Ventajas Operacionales:

1. **Adiós a `<ng-template>`:** La necesidad de recurrir a la etiqueta `<ng-template>` para definir la parte *else* o para estructuras condicionales anidadas o complejas desaparece. Esto limpia significativamente el código HTML, lo hace más directo y elimina una capa de abstracción.
2. **Lógica Anidada Simplificada:** La sintaxis permite encadenar múltiples condiciones de una manera natural y legible, simulando la funcionalidad de un `else if`, aunque típicamente se logra con `@switch` para múltiples casos, la anidación de `@if` es más limpia.
3. **Mejor Rendimiento:** Estas nuevas directivas estructurales están optimizadas para la compilación, facilitando al compilador la identificación y el *tree-shaking* de bloques no utilizados, y potencialmente resultando en *bundles* más pequeños y un renderizado inicial más rápido.

La directiva `@for`

La directiva `@for` es una de las características más significativas introducidas en Angular para modernizar la sintaxis de las plantillas (templates). Es la versión modernizada, simplificada y más eficiente de la directiva estructural `*ngFor` (la cual es un *micro-sintaxis*). Su propósito principal sigue siendo el mismo: iterar sobre colecciones de datos, como listas o arreglos (arrays), en el template y generar un bloque de código HTML dinámicamente para cada ítem de la colección.

El cambio a la sintaxis `@for` responde a la necesidad de mejorar el rendimiento en la detección de cambios y hacer que la lógica de bucle sea más legible y expresiva dentro del marcado HTML. Además, fuerza una mejor práctica al requerir la cláusula `@track`, esencial para una óptima gestión de la identidad de los elementos por parte del framework.

Propósito y Ventajas:

1. **Sintaxis Integrada al Template:** Utiliza el nuevo *control flow* de Angular basado en bloques, lo que elimina la necesidad del prefijo `*` de las directivas estructurales

y hace que la estructura del template sea más parecida a un código de control de flujo estándar.

2. **Rendimiento Mejorado:** Al estar optimizada por el *runtime* de Angular, la directiva `@for` ofrece un mejor rendimiento, especialmente en listas grandes, en comparación con `*ngFor`.
3. **Manejo Explícito de la Clave de Seguimiento (@track):** La cláusula `@track` (equivalente al `trackBy` de `*ngFor`) es obligatoria en `@for`. Esto asegura que los desarrolladores siempre especifiquen cómo Angular debe identificar un ítem de manera única. Esto es fundamental para la eficiencia, ya que permite a Angular mover o actualizar elementos del DOM de forma selectiva en lugar de recrearlos todos.

Si se quiere utilizar el ítem completo como clave, se puede usar `@track $implicit`.

Si se quiere usar el índice, se puede usar `@track $index` (aunque no se recomienda si la lista puede cambiar de orden, ya que es menos eficiente).

4. **Soporte Nativo para la Cláusula @empty:** A diferencia de `*ngFor` que requería una directiva `*ngIf` adicional para manejar el estado de una lista vacía, `@for` incluye una cláusula `@empty` nativa. Este bloque se renderiza automáticamente si la colección es nula o vacía.

Sintaxis Básica:

```
@for (item of items; track item.id) {  
  <div>{{ item.name }}</div>  
}
```

Sintaxis Completa (con Opcionales):

```
@for (item of items; track item.id; let i = $index; let isEven = $even) {  
  <div [class.even]="isEven">  
    {{ i + 1 }}. {{ item.name }}  
  </div>  
} @empty {  
  <div>No hay elementos para mostrar.</div>  
}
```

Variables de Contexto Adicionales (Variables Locales):

Al igual que `*ngFor`, `@for` expone variables de contexto que pueden ser utilizadas dentro del bloque:

Variable	Descripción
<code>\$index</code>	El índice de la iteración actual (0-basado).
<code>\$first</code>	<code>true</code> si es el primer ítem en el arreglo.
<code>\$last</code>	<code>true</code> si es el último ítem en el arreglo.
<code>\$even</code>	<code>true</code> si el índice es un número par.
<code>\$odd</code>	<code>true</code> si el índice es un número impar.
<code>\$implicit</code>	El ítem actual en la iteración (equivalente a <code>item</code> en la sintaxis).

Estas variables se acceden usando la sintaxis

```
let variableName = $variableKeyword
```

dentro de la definición de `@for`.

@switch y @case: Una Directiva Estructural Moderna y Expresiva

La introducción de las directivas `@switch` y `@case` en Angular representa una mejora significativa en la forma en que los desarrolladores manejan la renderización condicional basada en múltiples opciones, ofreciendo una sintaxis más limpia, intuitiva y expresiva en comparación con la directiva estructural tradicional `*ngSwitch` y sus compañeras `*ngSwitchCase` y `*ngSwitchDefault`.

Propósito y Ventajas

La directiva `@switch` actúa como un mecanismo de selección para determinar qué bloque de código HTML debe ser renderizado en el *Document Object Model* (DOM) basándose en la evaluación de una única expresión.

Ventajas clave sobre `*ngSwitch`:

- 1. Sintaxis Concisa y Legible:** La estructura `@switch(...)` { ... } es inherentemente más fácil de leer y se asemeja más a la estructura de un `switch` en JavaScript o TypeScript, mejorando la comprensión del flujo de control en la plantilla.
- 2. Rendimiento:** Al ser un *bloque de control* nativo de la sintaxis de plantillas de Angular (similar a `@if` y `@for`), `@switch` está optimizado para la detección de cambios y la manipulación del DOM, lo que puede resultar en un rendimiento ligeramente mejorado en comparación con las directivas estructurales más antiguas.

3. **Encapsulación y Claridad:** Evita la necesidad de anidar múltiples elementos HTML (como `<div>` o `<ng-container>`) para aplicar las directivas `*ngSwitchCase` y `*ngSwitchDefault`, reduciendo la complejidad del código de la plantilla.

Sintaxis Detallada

La estructura básica del bloque `@switch` se define de la siguiente manera:

```
@switch (expresionAevaluar) {  
  @case (valor1) {  
    <p>Opción uno seleccionada.</p>  
  }  
  @case (valor2) {  
    <p>Opción dos seleccionada.</p>  
  }  
  @default {  
    <p>Ninguna de las opciones anteriores coincide.</p>  
  }  
}
```

Explicación de las Partes:

- **@switch (expresionAevaluar):**
 - Este es el bloque contenedor y define la expresión cuyo valor será evaluado y comparado.
 - La expresión puede ser una propiedad del componente, el resultado de una función o cualquier expresión de plantilla válida (por ejemplo, `userRole`, `statusCode`, `getTheme()`).
- **@case (valor):**
 - Los bloques `@case` son las condiciones. La expresión dentro del paréntesis (`valor`) se compara estrictamente (`===`) con el valor de la `expresionAevaluar` definida en el `@switch`.
 - El primer bloque `@case` que coincida será el único contenido renderizado dentro del DOM.
 - Se pueden utilizar múltiples `@case` para una misma expresión.
- **@default:**
 - Este bloque es opcional, pero altamente recomendado para manejar escenarios en los que la `expresionAevaluar` no coincide con ninguna de las opciones `@case` proporcionadas.

- Funciona como la cláusula default en una sentencia switch tradicional. Solo se ejecuta si no se encuentra ninguna coincidencia anterior.

Ejemplo Práctico

Consideremos un componente que necesita mostrar diferentes mensajes de estado según el valor de una variable `estadoCarga`:

Componente TypeScript:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-estado-carga',
  template: `...` // Template definido abajo
})
export class EstadoCargaComponent {
  estadoCarga: 'cargando' | 'exitoso' | 'error' | 'pendiente' = 'cargando';

  // Método para cambiar el estado (para propósitos de ejemplo)
  simularCarga() {
    setTimeout(() => this.estadoCarga = 'exitoso', 3000);
  }
}

Plantilla (Template HTML) usando @switch:
<div>
  <h2>Estado de la Operación:</h2>

  @switch (estadoCarga) {
    @case ('cargando') {
      <div class="alert alert-info">
        <i class="fas fa-spinner fa-spin"></i>
        Procesando su solicitud. Por favor, espere...
      </div>
    }
    @case ('exitoso') {
      <div class="alert alert-success">
        <i class="fas fa-check-circle"></i>
        ¡Operación completada con éxito!
      </div>
    }
    @case ('error') {
      <div class="alert alert-danger">
        <i class="fas fa-times-circle"></i>
        Ha ocurrido un error. Intente de nuevo más tarde.
      </div>
    }
    @default {
      <div class="alert alert-warning">
        Estado desconocido: {{ estadoCarga }}.
      </div>
    }
  }

  <button (click)="simularCarga()">Cambiar a Exitoso</button>
</div>
```

La introducción de `@if`, `@for` y `@switch` en Angular 18 representa un paso audaz hacia una sintaxis de *template* más concisa, potente y, fundamentalmente, más amigable para el desarrollador. Al adoptar una sintaxis más cercana a la lógica de programación estándar y al exigir optimizaciones de rendimiento como el `track` en `@for`, Angular no solo mejora la experiencia de codificación sino que también eleva el estándar de rendimiento en las aplicaciones construidas con el *framework*. Si bien las directivas tradicionales aún son compatibles por razones de retrocompatibilidad, el uso de las nuevas directivas es la práctica recomendada para todos los nuevos desarrollos y para la modernización de código existente.

Pipes en Angular: Transformación de Datos en las Plantillas

Los **Pipes** son una característica fundamental y poderosa en Angular que ofrece un mecanismo elegante y altamente legible para transformar y formatear datos directamente dentro de las plantillas HTML (vistas). Su propósito principal es tomar un valor de entrada, aplicar una lógica de transformación y devolver un valor de salida formateado, sin modificar el dato original en el componente. Esto contribuye a una separación clara de responsabilidades: el componente gestiona la lógica de la aplicación y los datos, y el pipe se encarga de la presentación de esos datos.

Pipes Nativos (Built-in) de Angular

Angular proporciona un conjunto robusto de pipes predefinidos listos para usar en cualquier proyecto. Estos cubren las necesidades de formateo más comunes:

Pipes de Transformación de Cadenas de Texto

- **UpperCasePipe**: Convierte toda la cadena de texto de entrada a mayúsculas.

Ejemplo de uso: `{{ 'texto a transformar' | uppercase }}` -> **TEXTO A TRANSFORMAR**

- **LowerCasePipe**: Convierte toda la cadena de texto de entrada a minúsculas.

Ejemplo de uso: `{{ 'TEXTO A TRANSFORMAR' | lowercase }}` -> **texto a transformar**

- **TitleCasePipe**: Convierte la primera letra de cada palabra de la cadena a mayúsculas.

Ejemplo de uso: `{{ 'angular es genial' | titlecase }}` -> **Angular Es Genial**

Pipes de Fechas y Horas

- **DatePipe:** Esencial para formatear variables de tipo **Date** o cadenas de fecha (ISO 8601) en diferentes formatos legibles por el usuario, basándose en la configuración local (locale) de la aplicación.

Ejemplo de uso básico: `{{ fechaHoy | date }}`

Ejemplo con formato preestablecido: `{{ fechaHoy | date:'short' }}`

Ejemplo con formato personalizado: `{{ fechaHoy | date:'dd/MM/yyyy HH:mm:ss' }}`

Pipes Numéricos y Financieros

- **CurrencyPipe:** Utilizado para formatear valores numéricos como representaciones de moneda. Permite especificar el código de la moneda (ej. 'USD', 'EUR', 'ARS'), si el símbolo debe mostrarse y la precisión decimal.

Ejemplo de uso: `{{ precio | currency:'EUR':'symbol':'1.2-2' }}`

- **DecimalPipe (o NumberPipe):** Permite un control preciso sobre el formato de números decimales, incluyendo el número mínimo y máximo de dígitos enteros y fraccionarios a mostrar.

Sintaxis: `{{ valor | number[:dígitosEnteros:dígitosFraccionarios] }}`

Descomposición de la Sintaxis de **DecimalPipe**:

- **valor:** La variable de tipo numérico a la que se aplicará el formato.
- **number:** El nombre del pipe (**DecimalPipe** o su alias **number**).
- **dígitosEnteros:** Especifica la cantidad mínima de dígitos enteros que se mostrarán. Puede ser un número o un rango.

Formato: **minimo-maximo** (ej. '1-5'). Si el número tiene menos del mínimo, se rellena con ceros a la izquierda.

- **dígitosFraccionarios:** Especifica el número mínimo y máximo de dígitos decimales.

Formato: **minimo-maximo** (ej. '2-2'). El número se redondea si excede el máximo.

Ejemplo de uso: `{{ 123.456 | number:'1.2-2' }}` -> **123.46** (mínimo 1 entero, 2 decimales fijos)

Pipes de Manipulación de Colecciones y Subcadenas

- **SlicePipe:** Similar al método `slice()` disponible para arrays y cadenas de texto en JavaScript. Permite seleccionar un subconjunto de elementos de una lista o una subcadena de texto, especificando un índice de inicio y (opcionalmente) uno de fin.
 - *Ejemplo con cadena:* `{{ 'AngularFramework' | slice:0:7 }}` -> **Angular**
 - *Ejemplo con array:* `{{ listaProductos | slice:3:6 }}` (Devuelve los elementos del índice 3, 4 y 5)
- **JsonPipe:** Útil para la depuración, convierte un objeto o array de JavaScript en una cadena JSON formateada.
 - *Ejemplo de uso:* `{{ objetoComplejo | json }}`

Pipes Personalizados (Custom Pipes)

Aunque los pipes nativos cubren muchas necesidades, la verdadera flexibilidad se encuentra en la capacidad de crear pipes personalizados para encapsular lógica de transformación específica del negocio o de la aplicación.

Creación de un Pipe Personalizado:

Generación: Se utiliza la CLI de Angular para generar el esqueleto del pipe.
`ng generate pipe <nombre-del-pipe>`.

Esto crea un archivo TypeScript con la interfaz `PipeTransform`.

Implementación: El pipe debe implementar la interfaz `PipeTransform` y definir el método `transform`.

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'capitalizar'
})
export class CapitalizarPipe implements PipeTransform {
  // El método 'transform' recibe el valor de entrada y argumentos opcionales.
  transform(value: string, todas: boolean = true): string {
    // Lógica de transformación
    if (todas) {
      // Capitaliza todas las palabras
      return value.split(' ').map(palabra => palabra.charAt(0).toUpperCase() +
palabra.slice(1).toLowerCase()).join(' ');
    } else {
      // Capitaliza solo la primera palabra
      return value.charAt(0).toUpperCase() + value.slice(1).toLowerCase();
    }
  }
}
```

Utilización de un Pipe Personalizado:

Módulos (versiones antiguas): El pipe debe ser declarado en el `declarations` array del módulo que lo utiliza.

Componentes Standalone (versiones modernas): En los componentes `standalone`, es **obligatorio** importarlo explícitamente en el array `imports` del `@Component` que lo utiliza.

```
import { Component } from '@angular/core';
import { CapitalizarPipe } from './capitalizar.pipe'; // Importación necesaria

@Component({
  standalone: true,
  selector: 'app-ejemplo',
  template: `
    <p>{{ 'hola mundo' | capitalizar }}</p>
    <p>{{ 'otra frase' | capitalizar:false }}</p>
  `,
  imports: [CapitalizarPipe] // Importación explícita
})
export class EjemploComponent {}
```

Errores Comunes:

- **Pipe not found (o error de plantilla similar):** Este es el error más frecuente y ocurre cuando el pipe (ya sea nativo o personalizado) no ha sido importado correctamente.

Solución: Asegurarse de que el pipe esté declarado en el módulo correspondiente o, en el caso de componentes `standalone`, esté listado en el array `imports: [...]` del decorador `@Component`. Para pipes nativos como `DatePipe` o `CurrencyPipe`, a menudo están disponibles automáticamente si se importa el `CommonModule`, o deben importarse individualmente en componentes `standalone`.

Routing en Angular: Navegación Potente en Single Page Applications (SPA)

El sistema de routing es la columna vertebral de cualquier Single Page Application (SPA) desarrollada con Angular. Su función principal es gestionar la navegación entre las distintas vistas o estados de la aplicación, imitando el comportamiento de un sitio web tradicional con múltiples páginas, pero sin la necesidad de recargar la página completa. Esto resulta en una experiencia de usuario más fluida y rápida. Cada segmento de la URL se mapea internamente a un componente específico de Angular, el cual es dinámicamente insertado en la interfaz.

En las versiones modernas de Angular (a partir de la v17 con la introducción de aplicaciones *standalone*), la configuración del routing se ha simplificado significativamente, migrando de los engorrosos **NgModules** a un enfoque basado en funciones. La función clave para inicializar este servicio es `provideRouter()`, que se registra en la configuración del *bootstrap* de la aplicación. **Conceptos Fundamentales y Directivas Esenciales del Router**

Para implementar el routing de manera efectiva, es crucial comprender los siguientes elementos:

- **Routes:** Es el array central de configuración. Define la **tabla de enrutamiento**, estableciendo la relación estricta entre una ruta (el *path* de la URL) y el **componente** que Angular debe instanciar y renderizar. Cada objeto *Route* puede especificar el *path*, el *component* asociado, *children* (rutas anidadas) y la estrategia de carga (*lazy loading*).
- **router-outlet:** Esta directiva es el **marcador de posición** (*placeholder*) en la plantilla HTML (típicamente en `app.component.html`). Actúa como el *view container* donde Angular inyecta dinámicamente el componente correspondiente a la ruta que está activa en ese momento. Sin esta directiva, la navegación ocurre, pero el componente no tiene dónde mostrarse.
- **routerLink:** Es la directiva de navegación declarativa, que se aplica a elementos interactivos como anclas (`<a>`) o botones. Permite cambiar la ruta de la aplicación **sin forzar una recarga completa del navegador**, manteniendo la SPA. Es la alternativa preferida al uso directo del atributo `href`.
- **routerLinkActive:** Directiva complementaria a `routerLink`. Añade automáticamente una o más **clases CSS** (ej. `active`, `seleccionado`) al elemento host cuando la ruta a la que apunta el `routerLink` está activa o es ancestro de la ruta actual. Esto es esencial para **resaltar visualmente** el enlace activo en menús de navegación.

- **provideRouter()**: La función moderna (v17+) para registrar el servicio de routing a nivel de la aplicación *standalone*. Reemplaza el patrón anterior de **RouterModule.forRoot()**, haciendo la configuración más directa y *treeshakeable* (eliminando código no usado).

Implementación y Configuración Detallada del Routing

La configuración del routing en Angular se realiza en tres pasos principales, promoviendo la separación de responsabilidades:

Paso 1: Definición de la Tabla de Rutas (**app.routes.ts**):

Se crea un archivo específico para centralizar todas las definiciones. Es un array de objetos *Route* que mapean la URL al componente:

```
import { Routes } from '@angular/router';
import { HomeComponent } from '../home/home.component';
import { ProductsComponent } from '../products/products.component';

export const routes: Routes = [
  { path: 'home', component: HomeComponent, title: 'Inicio' },
  { path: 'products', component: ProductsComponent, title: 'Productos' },
  { path: '', redirectTo: '/home', pathMatch: 'full' }, // Redirección por defecto
  { path: '**', loadComponent: () => import('../not-found/not-found.component').then(c => c.NotFoundComponent) } // Wildcard
];
```

- **Ruta Wildcard (path: '**')**: Esta ruta debe ser **obligatoria** y **siempre la última** en el array. Actúa como *fallback*, capturando cualquier URL que no haya coincidido con las rutas definidas previamente. Típicamente se usa para redirigir al inicio o mostrar un componente de error 404 (Página no encontrada).

Paso 2: Registro de las Rutas (**app.config.ts**):

En el archivo de configuración principal, se registra el servicio de routing utilizando la función **provideRouter()** y pasando el array de rutas:

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from '../app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes) // Registro del servicio de routing con las rutas definidas
  ]
};
```

Paso 3: Inserción del router-outlet (app.component.html):

Se coloca la directiva en el *template* del componente raíz para indicar dónde se deben cargar los componentes de la ruta:

```
<nav>
  <a routerLink="/home" routerLinkActive="active">Inicio</a>
  <a routerLink="/products" routerLinkActive="active">Productos</a>
</nav>

<router-outlet></router-outlet>
```

Importante para Componentes Standalone: Dado que los componentes *standalone* no dependen de NgModules, las directivas `RouterOutlet` y `RouterLink` no están disponibles automáticamente. Es necesario importarlos explícitamente en el array `imports` del decorador `@Component`: `imports: [RouterOutlet, RouterLink]`.

Rutas Dinámicas y Parámetros de Ruta

Las aplicaciones modernas requieren rutas que contengan información variable, como el ID de un producto o un usuario.

Sintaxis de Ruta Dinámica:

Se utiliza el prefijo de dos puntos (`:`) para definir un segmento de URL como un parámetro:

```
{ path: 'products/:id', loadComponent: () => import('./product-detail/product-
detail.component').then(c => c.ProductDetailComponent) }
```

Acceso a Parámetros con `withComponentInputBinding()` (Recomendado Angular 16+):

Con la opción `withComponentInputBinding()`, el router automáticamente mapea los parámetros de ruta (ej. `:id`) y los *query params* a propiedades `@Input` del componente de destino. Esto elimina la necesidad de inyectar el servicio `ActivatedRoute`, simplificando enormemente el componente:

En `app.config.ts`:

```
provideRouter(routes, withComponentInputBinding())
```

En el componente (`ProductDetailComponent`):

```
import { Component, Input } from '@angular/core';

@Component({ ... })
export class ProductDetailComponent {
  @Input() id!: string; // El nombre de la propiedad debe coincidir con el parámetro :id

  ngOnInit() {
    console.log('ID del producto:', this.id);
  }
}
```

Lazy Loading (Carga Diferida)

El *Lazy Loading* es una técnica de optimización crítica para grandes aplicaciones. Permite **cargar módulos o componentes JavaScript solo cuando el usuario navega a la ruta asociada**, en lugar de incluirlos en el *bundle* inicial. Esto reduce drásticamente el tiempo de carga inicial de la aplicación.

En aplicaciones *standalone*, se utiliza la sintaxis `loadComponent` para el *lazy loading* a nivel de componente:

```
export const routes: Routes = [
  // Carga inmediata
  { path: 'home', component: HomeComponent },

  // Lazy Loading: El componente 'AdminDashboardComponent' y su JS se cargan solo al acceder a /admin
  { path: 'admin', loadComponent: () => import('./admin/admin-dashboard.component').then(c => c.AdminDashboardComponent) },

  // Lazy Loading de un conjunto de rutas hijas (típico para módulos funcionales)
  {
    path: 'checkout',
    loadChildren: () => import('./checkout/checkout.routes').then(mod => mod.CHECKOUT_ROUTES)
  }
];
```

Guards (Protección de Rutas)

Los *Guards* son interfaces o funciones que el router invoca antes de permitir o denegar el acceso a una ruta o la salida de una ruta. Son esenciales para implementar seguridad (ej. solo usuarios autenticados) o prevenir la pérdida de datos (ej. formularios no guardados).

En Angular moderno, se prefieren los **guards funcionales** (`CanActivateFn`):

```
import { CanActivateFn, Router } from '@angular/router';
import { inject } from '@angular/core';
import { AuthService } from './auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isAuthenticated()) {
    return true; // Permite el acceso
  } else {
    // Redirige a la página de login
    return router.createUrlTree(['/login']);
  }
};
```

Uso en la Definición de Ruta:

```
{
  path: 'profile',
  loadComponent: () => import('./user/profile.component').then(c => c.ProfileComponent),
  canActivate: [authGuard] // Aplicación del guard
}
```

Un *guard* funcional debe retornar un **boolean** (**true** para permitir, **false** para denegar), un **UrlTree** (para forzar una redirección) o un **Observable/Promise** de estos tipos.

Navegación Programática (Imperativa)

Aunque **routerLink** es ideal para la navegación declarativa, a menudo se requiere iniciar la navegación desde el código TypeScript (ej. después de un *login* exitoso o al hacer clic en un botón sin **routerLink**). Esto se logra inyectando el servicio **Router**:

```
import { Router } from '@angular/router';
import { Component } from '@angular/core';

@Component({ ... })
export class LoginComponent {
  constructor(private router: Router) {}

  login() {
    // Lógica de autenticación ...
    if (success) {
      // Navegación absoluta a /dashboard
      this.router.navigate(['/dashboard']);

      // Navegación relativa (útil en rutas anidadas)
      // this.router.navigate(['../'], { relativeTo: this.activatedRoute });
    }
  }

  goToProduct(id: number) {
    // Navegación con parámetros
    this.router.navigate(['/products', id]);
  }
}
```

Opciones Avanzadas del Router (`withFeatures`)

La función `provideRouter()` acepta un segundo argumento que permite configurar *features* avanzadas, mejorando el rendimiento y la usabilidad:

- `withComponentInputBinding()`: Ya mencionado, es fundamental para simplificar el acceso a parámetros de ruta.
- `withPreloading(PreloadAllModules)`: Habilita la **precarga de módulos lazy en segundo plano**. Después de que la aplicación inicial carga, el router precarga los archivos JS de las rutas *lazy*, de modo que cuando el usuario navega a ellas, la transición es casi instantánea.
- `withDebugTracing()`: Activa un **registro detallado (logs)** de todos los eventos del ciclo de vida del router en la consola del navegador. Extremadamente útil para depurar problemas de navegación o *guards*.
- `withInMemoryScrolling({ scrollPositionRestoration: 'enabled' })`: Mejora la experiencia del usuario al **restaurar la posición de scroll** al navegar hacia atrás o adelante, imitando el comportamiento de un navegador tradicional.
- `withRouterConfig({ useHash: true })`: Cambia la estrategia de ubicación de la ruta para usar el *hash* (#) en la URL (ej. `[midominio.com/#/home](https://midominio.com/#/home)`). Esto es necesario cuando el servidor no puede ser configurado para manejar URLs limpias (*clean URLs*).

Ejemplo de Configuración Completa:

```
provideRouter(  
  routes,  
  withDebugTracing(),  
  withComponentInputBinding(),  
  withPreloading(PreloadAllModules),  
  withInMemoryScrolling({ scrollPositionRestoration: 'enabled' })  
)
```

Diagnóstico de Errores Comunes y Troubleshooting

1. **Error: Cannot match any routes**: El router no encontró una coincidencia para la URL.

Solución: Revisar la URL en el navegador. Asegurarse de que el *path* existe en `app.routes.ts`. Verificar que la ruta *wildcard* (`path: '**'`) esté presente y sea la última.

2. **El Componente no se Renderiza**: La navegación ocurre, pero la vista permanece vacía.

Solución: Confirmar que la directiva `<router-outlet>` está incluida en el *template* del componente principal (`app.component.html`).

3. **Olvidar Importar Directivas Standalone:** Error de plantilla (`Can't bind to 'routerLink' since it isn't a known property...`).

Solución: En componentes *standalone*, agregar `RouterOutlet` y `RouterLink` al array `imports: [...]` del decorador `@Component`.

4. **Guard no Redirige Correctamente:** El *guard* retorna `false`, pero el usuario se queda en la ruta prohibida.

Solución: Si el acceso se deniega, el *guard* debe retornar explícitamente un `UrlTree` (usando `router.createUrlTree(['/login'])`) para forzar la redirección y proporcionar *feedback* al usuario.

5. **Lazy Loading Falla (Cannot find module):** El *bundler* (Webpack/Vite) no puede localizar el archivo del módulo *lazy*.

Solución: Verificar la ruta del `import()` dinámico. Asegurarse de que el componente o el array de rutas esté exportado correctamente desde su archivo.

Buenas Prácticas de Routing en Angular 2026

1. **Priorizar `provideRouter()`:** Usar esta función para la configuración inicial en todas las nuevas aplicaciones *standalone*.
2. **Centralización de Rutas:** Mantener la tabla de rutas principal (`routes: Routes`) en un archivo dedicado (`app.routes.ts`).
3. **Lazy Loading por Defecto:** Aplicar `loadComponent` o `loadChildren` a todas las rutas que no sean absolutamente esenciales para la carga inicial.
4. **Guards Funcionales:** Utilizar la sintaxis de *guards* funcionales (`CanActivateFn`) inyectando servicios con `inject()`, en lugar de los antiguos *guards* basados en clases.
5. **Habilitar Input Binding:** Activar `withComponentInputBinding()` para un acceso limpio y tipado a los parámetros de ruta a través de `@Input()`.
6. **Manejo de 404:** Definir siempre una ruta *wildcard* (`path: '**'`) al final del arreglo de rutas para ofrecer una experiencia de usuario consistente cuando se navega a una URL inexistente.

Comunicación entre Componentes en Angular

Una aplicación Angular se estructura como un árbol de componentes, donde cada uno maneja una parte de la interfaz. Para construir aplicaciones complejas, mantenibles y modulares, es fundamental que estos componentes puedan intercambiar información.

La comunicación se aborda principalmente en tres escenarios:

1. **De Padre a Hijo:** El componente contenedor (Padre) envía datos al componente contenido (Hijo) utilizando las propiedades de entrada (@Input).
2. **De Hijo a Padre:** El componente contenido (Hijo) notifica eventos o envía información al componente contenedor (Padre) mediante eventos de salida (@Output).
3. **Entre Componentes sin Relación Directa:** Para componentes que no son directamente padre e hijo (como “hermanos” o partes distantes del árbol), se emplean técnicas más avanzadas (como servicios o librerías de gestión de estado).

Dominar el uso de @Input y @Output es esencial para compartir información y diseñar el flujo de datos dentro de la aplicación.

1. Comunicación de Padre a Hijo con @Input en Angular

Las propiedades de entrada, marcadas con el decorador @Input(), son el mecanismo principal en Angular para permitir que un componente padre pase datos a un componente hijo. Este flujo de datos es **unidireccional** (del padre al hijo), manteniendo una arquitectura clara y un patrón de flujo de datos que desciende por el árbol de componentes.

Flujo y Mecanismo:

1. **Definición en el Componente Hijo:** En el componente hijo, se define una propiedad de entrada usando @Input(). Esta propiedad es la “puerta” por donde el hijo recibirá los datos.
2. **Uso en el Componente Padre:** El componente padre envía el valor a la propiedad de entrada del hijo mediante el **property binding** (uso de corchetes [] en la plantilla del padre).
3. **Propagación del Cambio (Internamente):**
 - El padre actualiza su estado (el valor que debe pasar).
 - Angular detecta este cambio.
 - El nuevo valor se propaga automáticamente a la propiedad @Input() del componente hijo.
 - El componente hijo vuelve a renderizar su vista si es necesario, mostrando el dato actualizado.

Pasos Clave:

1. Definir la Propiedad de Entrada en el Componente Hijo:

En el componente hijo (HijoComponent), utiliza el decorador `@Input()`:

```
@Input() nombre: string = '';
```

2. Propagar el Valor desde el Componente Padre:

En el componente padre, enlaza la propiedad de entrada del hijo (`[nombrePropiedad]`) a una variable o expresión del padre.

```
<app-hijo [nombre]="Juan"></app-hijo>
```

Diagrama:



2. Comunicación de Hijo a Padre: `@Output` y `EventEmitter`

Cuando la interacción del usuario o un cambio interno ocurre dentro de un componente hijo, este a menudo necesita informar al componente padre que algo relevante ha sucedido. Sin embargo, en Angular, la jerarquía de datos es estrictamente unidireccional por diseño (del padre al hijo). Por lo tanto, el hijo no puede modificar directamente el estado o las propiedades del padre. En su lugar, el hijo utiliza un mecanismo de **emisión de eventos** para notificar al padre.

Este patrón se logra mediante el uso conjunto del decorador `@Output()` y la clase **EventEmitter**. Es un mecanismo de “publicación/suscripción” que es fundamental para mantener el principio de **desacoplamiento** en la arquitectura de componentes. El hijo actúa como el “publicador” del evento, simplemente comunicando que algo ha ocurrido sin tener conocimiento de cómo reaccionará el padre. El padre, por su parte, actúa como el “suscriptor”, decidiendo qué acción tomar en respuesta al evento.

Principios Clave y Beneficios

1. **Unidireccionalidad Reforzada:** Mantiene el flujo de datos principal de Padre a Hijo, mientras que la comunicación inversa se maneja estrictamente a través de eventos, evitando mutaciones directas y haciendo más predecible el estado de la aplicación.
2. **Desacoplamiento:** El componente hijo no tiene dependencias directas del componente padre. Simplemente define un canal de comunicación de salida. Esto permite que el hijo sea reutilizado fácilmente en cualquier otro componente padre que sepa cómo manejar ese evento específico.
3. **Flexibilidad:** Permite al padre reaccionar al evento de diversas maneras: actualizar su propio estado, llamar a un servicio, interactuar con otros componentes, o incluso reaccionar de forma diferente dependiendo de los datos enviados por el hijo.

Flujo Detallado y Mecanismos de Angular

El proceso de comunicación de Hijo a Padre implica tres etapas claras:

Comunicación de Hijo a Padre (Emisión de Eventos)

- **Definición (Componente Hijo):** El hijo declara una propiedad con el decorador `@Output()` que debe ser una instancia de `EventEmitter<T>`, donde T es el tipo de dato que se enviará (opcionalmente). El nombre de esta propiedad será el nombre del evento.
- **Escucha/Binding (Componente Padre):** En la plantilla del padre, al insertar el selector del hijo, se utiliza la sintaxis de *event binding*
`((nombreDelEvento)="métodoDelPadre($event)")` para 'engancharse' a la escucha.
- **Emisión (Componente Hijo):** En respuesta a una acción interna (p. ej., (click) en un botón dentro del hijo), el hijo llama al método `.emit()` de su propiedad `@Output`, enviando opcionalmente los datos que el padre necesita.
- **Reacción (Componente Padre):** El método asociado en la clase del padre (ej. `métodoDelPadre`) se ejecuta automáticamente, recibiendo los datos del hijo a través de la variable especial `$event`.

Implementación Paso a Paso con Código

Para ilustrar este mecanismo, consideremos un `HijoComponent` que tiene un botón y un `PadreComponent` que debe mostrar un mensaje cuando se pulsa ese botón.

- **Definir el Evento de Salida en el Componente Hijo (HijoComponent):**

Dentro de la clase del componente hijo, importamos `Output` y `EventEmitter`. Definimos la propiedad del evento de salida, asegurándonos de inicializarla con una nueva instancia de `EventEmitter`.

```
import { Component, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-hijo',
  template: `
    <button (click)="notificarAlPadre()">Notificar al Padre</button>
  `
})
export class HijoComponent {
  // @Output expone la propiedad como un evento.
  // <string> indica que el evento emitirá un dato de tipo string.
  @Output() eventoEmitido: EventEmitter<string> = new EventEmitter<string>();

  // Este método se dispara al hacer clic en el botón interno.
  notificarAlPadre(): void {
    const mensajeDeInteraccion = `¡Hola Padre! Mi estado interno ha cambiado (clic realizado).`;

    // **Punto Crucial:** El hijo emite el evento, pasando los datos al suscriptor (el padre).
    this.eventoEmitido.emit(mensajeDeInteraccion);

    console.log('Hijo: Evento emitido con el mensaje:', mensajeDeInteraccion);
  }
}
```

- **Capturar y Manejar el Evento en el Componente Padre (PadreComponent):**

El padre es responsable de escuchar el evento (eventoEmitido) y definir cómo reaccionar.

- **Plantilla del Padre (padre.component.html):**

El nombre de la propiedad @Output del hijo (eventoEmitido) se utiliza en la plantilla del padre entre paréntesis (). El objeto especial \$event contiene los datos pasados durante la llamada a .emit().

```
<div>
  <h2>Componente Padre</h2>
  <p>Mensaje recibido del hijo: <strong>{{ mensajeDelHijo || 'Esperando notificación...' }}</strong></p>

  <app-hijo (eventoEmitido)="manejarNotificacion($event)"></app-hijo>

</div>
```

- **Clase del Padre (PadreComponent.ts):**

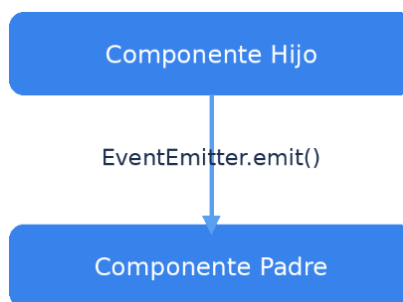
La clase del padre implementa el método que reacciona a la notificación.

```
@Component({
  selector: 'app-padre',
  // ...
})
export class PadreComponent {
  mensajeDelHijo: string = '';

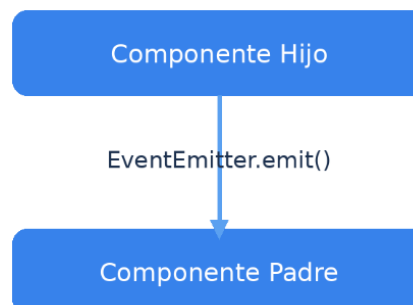
  // El método recibe los datos (el string) que el hijo emitió.
  manejarNotificacion(datosRecibidos: string): void {
    // El padre actualiza su propio estado basándose en la información del hijo.
    this.mensajeDelHijo = datosRecibidos;
    console.log('Padre: Evento capturado exitosamente. Datos:', datosRecibidos);

    // Aquí el padre podría ejecutar lógica de negocio, llamar a un servicio, etc.
  }
}
```

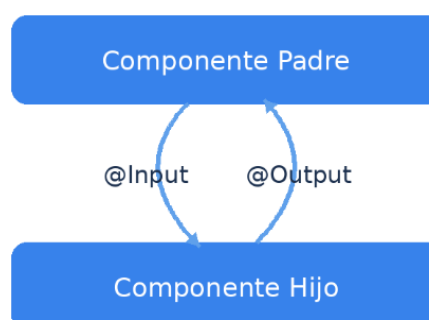
Diagrama de Flujo Conceptual (Hijo → Padre)



Flujo completo Padre ↔ Hijo



En la práctica, muchas veces ambos mecanismos se combinan.



- El padre envía datos.
- El hijo los usa.
- El hijo notifica cambios.
- El padre reacciona.

Descendente (Padre → Hijo): Se utiliza **@Input()** para inyectar datos, configuraciones, o funciones de *callback* desde el padre hacia el hijo. Esto define el estado inicial y la configuración del hijo.

Ascendente (Hijo → Padre): Se utiliza **@Output()** y **EventEmitter** para que el hijo notifique al padre sobre interacciones, cambios de estado internos, errores, o la finalización de tareas.

Al emplear esta estricta arquitectura de flujo de datos, los desarrolladores de Angular pueden construir aplicaciones complejas con componentes altamente reutilizables, mantenibles y fáciles de probar.

Servicios (Alternativa para Comunicación)

Otra forma de comunicación entre componentes es utilizando servicios para compartir datos y estado entre ellos. Los servicios actúan como contenedores de datos y lógica que se pueden inyectar en diferentes componentes. Esta es una forma efectiva de comunicar componentes que no tienen una relación directa de padre-hijo.

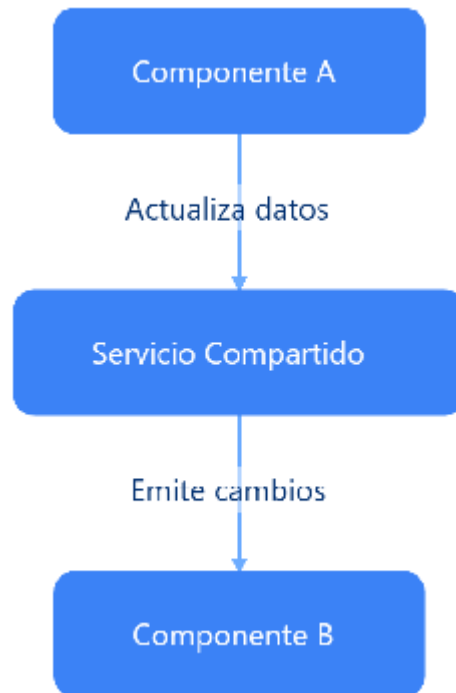
Recuerda que, en Angular, los cambios en las propiedades de entrada o en los eventos de salida en los componentes hijo se reflejarán automáticamente en el componente padre. La comunicación bidireccional es una de las características poderosas de Angular que permite crear aplicaciones interactivas y reactivas.

3. Comunicación entre componentes “hermanos” (Servicios)

Cuando dos componentes no están relacionados directamente en el árbol, no pueden comunicarse con **@Input** o **@Output**. En ese caso se utiliza un **servicio compartido** como intermediario.

El servicio actúa como un “centro de datos” o puente.

Esquema conceptual



Ambos componentes inyectan el mismo servicio (singleton). De esta manera:

- Componente A modifica datos en el servicio.
- Componente B observa esos cambios.

Angular actualiza la vista según corresponda.

Ejemplo simplificado

Servicio:

```
@Injectable({ providedIn: 'root' })
export class DatosService {
  mensaje = signal('Hola');
}
```

Componente A:

```
this.servicio.mensaje.set('Nuevo valor');
```

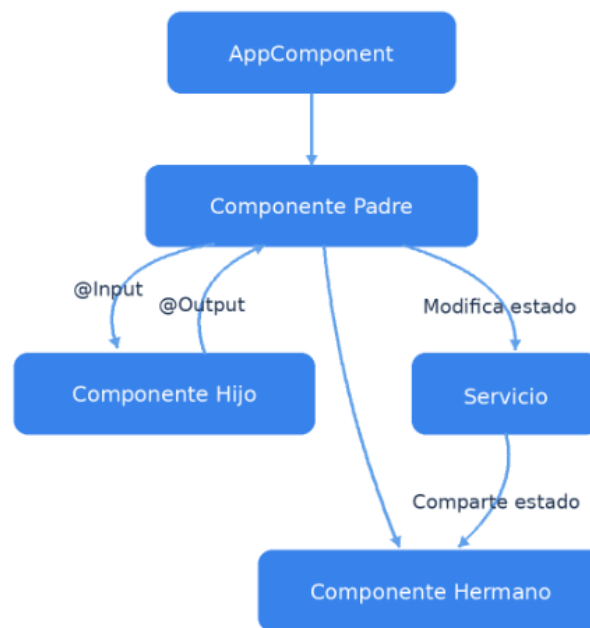
Componente B:

```
mensaje = this.servicio.mensaje;
```

Aquí los cambios se reflejan automáticamente gracias a la reactividad.

Modelo mental claro

Podemos resumir la comunicación así:



- Los datos bajan con **@Input**.
- Los eventos suben con **@Output**.
- Los servicios comparten estado horizontalmente.

Errores comunes y diagnóstico

1. Evento no capturado

Si el evento no funciona:

Verificar que el nombre del evento en el HTML coincida exactamente con el **@Output**.

Confirmar que se está llamando a **.emit()**.

2. Problemas de detección de cambios

Si se pasa un objeto por **@Input** y solo se modifica una propiedad interna, Angular puede no detectar el cambio dependiendo de la estrategia de detección.

Buenas prácticas modernas:

- Usar valores inmutables.
- Crear nuevos objetos en lugar de modificar los existentes.
- Utilizar valores inmutables y crear nuevos objetos cuando sea necesario.

Servicios e Inyección de Dependencias

Servicios

Son clases que se utilizan para organizar y compartir lógica, datos y funcionalidad comunes entre diferentes componentes de una aplicación. Los servicios desempeñan un papel fundamental en la arquitectura de una aplicación Angular, ya que permiten la separación de preocupaciones y promueven la reutilización de código.

Características:

- **Reutilización de código:** Puedes crear servicios para encapsular la lógica o la funcionalidad que se usa en varios lugares de tu aplicación. Esto promueve la reutilización del código y evita la duplicación.
- **Inyección de dependencias:** Angular tiene un sistema de inyección de dependencias incorporado que se utiliza para proporcionar instancias de servicios a componentes u otros servicios que los necesiten. Esto facilita la gestión de dependencias y asegura que los servicios se compartan de manera eficiente.
- **Singleton por defecto:** Cuando inyectas un servicio en un componente o en otro servicio, Angular crea una única instancia de ese servicio y la comparte en toda la aplicación. Esto asegura que los datos y la funcionalidad compartidos sean coherentes en toda la aplicación.
- **Separación de preocupaciones:** Los servicios ayudan a separar la lógica de negocios y la manipulación de datos de la presentación. Los componentes se centran en la interfaz de usuario y delegan tareas específicas a los servicios.
- **Interacción con datos externos:** Los servicios son comunes para interactuar con servidores, bases de datos o servicios web externos para recuperar o enviar datos. Pueden manejar solicitudes HTTP, WebSocket, almacenamiento local y otras operaciones de E/S.

Los servicios pueden actuar como intermediarios para la comunicación entre componentes. Pueden emitir eventos o utilizar Observables (a menudo con RxJS) para notificar a otros componentes sobre cambios o eventos importantes en la aplicación. Esto facilita la creación de aplicaciones interactivas y dinámicas.

Son útiles para gestionar la autenticación y la autorización en una aplicación. Puedes crear un servicio de autenticación que maneje la lógica de inicio de sesión, verificación de tokens y control de acceso, lo que permite que varios componentes accedan a esta funcionalidad de manera consistente.

En resumen, facilitan la organización del código, la separación de preocupaciones, la reutilización y la comunicación eficiente entre componentes. Además, permiten gestionar la

lógica de la aplicación, la configuración global y la interacción con recursos externos de manera eficiente y coherente.

Ejemplo:

Para crear un servicio se puede utilizar Angular CLI por medio del siguiente comando:

```
ng generate service nombre-del-servicio
```

Descomposición de comando:

- **ng:** hace referencia a Angular.
- **generate:** operación que desea realizar, en este caso generar o crear un elemento en angular, también puede escribirse con la letra “g” únicamente.
- **service:** elemento que desea generar, en este caso se está solicitando la creación de un servicio.
- **nombre-del-servicio:** como lo dice su nombre es el nombre que se le va a asignar al servicio en este caso. Al igual que en la creación de un proyecto los guiones indican la separación entre las palabras, Angular resuelve esto haciendo mayúscula la primera letra después de cada guion en los archivos de clase (se verá más adelante).

Suponiendo que se creó un servicio **persona** este es el código generado:

```
import { Injectable } from '@angular/core';

no usages
@Injectable({
  providedIn: root,
})
export class PersonaService {
  private personas: string[] = [];

  no usages
  agregarPersona(nombre: string) : void {
    this.personas.push(nombre);
  }

  no usages
  obtenerPersonas() {
    return this.personas;
  }
}
```

Una clase la cual contiene el decorador **@Injectable** que indica que dicho servicio va a poder ser inyectado como una dependencia en otros lugares de mi aplicación (por ejemplo: un componente, otro servicio, etc). Por defecto Angular añade la propiedad **provideIn: 'root'** que permite trabajar como singleton por lo que estará disponible en toda la aplicación sin necesidad de una configuración adicional.

Luego para utilizar dicho servicio es necesario inyectar en donde se necesite utilizarlo, en este caso se verá un ejemplo con un componente:

```
import { Component } from '@angular/core';
import { PersonaService } from '../persona.service';

no usages
@Component({
  selector: 'app-persona-listado',
  template: `
    <div> <h2>Personas</h2>
    <ul>
    @for(persona of personas; track persona.id){
      <li>{{ persona }}</li>
    }
    </ul>
    <input [(ngModel)]="nombre" placeholder="Nueva persona" />
    <button (click)="agregarPersona()">Agregar persona</button>
    </div>
  `,
})
export class PersonaComponent {
  personas: string[] = [];
  nombre: string = '';

  no usages
  constructor(private personaService: PersonaService) {
    this.personas = this.personaService.obtenerPersonas();
  }
  1+ usages
  agregarPersona() : void {
    this.personaService.agregarPersona(this.nombre);
    this.nombre = '';
  }
}
```

Para ello Angular utiliza un mecanismo de inyección desde el constructor por lo que es necesario declarar aquellas dependencias en el constructor junto con su modo de acceso. En caso de utilizarlo únicamente en el archivo typescript se puede declarar como privado. Si es necesario utilizarlo en el template tiene que ser declarado como `protected` o `public`, siempre es recomendable utilizar el modo de acceso más restringido o la mayor restricción posible para cumplir con uno de los principios **SOLID**.

En caso de no utilizar la propiedad **provideIn: 'root'** que ofrece el decorador **@Injectable** es necesario realizar una configuración adicional en el archivo de módulos. Donde se debe agregar en un arreglo llamado **providers** el servicio que se desea utilizar. En caso de no hacerlo Angular arrojará un mensaje de error diciendo que no se encuentra una dependencia para el servicio solicitado.

Ejemplo del error arrojado en consola: *No provider for ServiceName.*

A continuación, se muestra un ejemplo del código de ejemplo del servicio sin la propiedad **provideIn** y la configuración necesaria para que dicho servicio pueda ser utilizado a posterior.

- **persona.service.ts**

```
import { Injectable } from '@angular/core';

no usages
@Injectable()
export class PersonaService {
  private personas: string[] = [];

  no usages
  agregarPersona(nombre: string) : void {
    |   this.personas.push(nombre);
    |
  }

  no usages
  obtenerPersonas() {
    |   return this.personas;
    |
  }
}
```

- persona-component.ts

```
import { Component, OnInit } from '@angular/core';
import { PersonaService } from '../persona.service';

no usages
@Component({
  selector: 'app-persona-componente',

  providers: [PersonaService], // Solo accede el componente de persona
  standalone: true,
  template:
    `
    <div> <h2>Personas</h2>
    <ul>
    @for(persona of personas; track persona.id){
      <li>{{ persona }}</li>
    }
    </ul>
    <input [(ngModel)]="nombre" placeholder="Nueva persona" />
    <button (click)="agregarPersona()">Agregar persona</button>
    </div>
    `,
})
export class PersonaComponent {
  personas: string[] = [];
  nombre: string = '';

  no usages
  constructor(private personaService: PersonaService) {
    this.tasks = this.personaService.obtenerPersonas();
  }

  1+ usages
  agregarPersona() : void {
    this.personaService.agregarPersona(this.newTask);
    this.newTask = '';
  }
}
```

Al especificar providers: [PersonaService] en el decorador del componente, estás creando una instancia de PersonaService solo para PersonaComponente. Esto significa que no puedes usar PersonaService en otros componentes a menos que lo proporciones nuevamente en sus providers.

Esta es una técnica útil cuando quieres limitar el alcance de un servicio a un solo componente, lo que puede ayudar a mantener la separación de responsabilidades y mejorar la gestión de recursos en tu aplicación Angular. **Uso moderno con inject()**

En versiones modernas de Angular, es común utilizar la función **inject()** en lugar de la inyección por constructor. Esto permite un código más flexible y compatible con funciones fuera de clases.

Aquí Angular se encarga de proporcionar la instancia correcta del servicio.

Errores comunes y diagnóstico

- “No provider for Service!”

Ocurre cuando el servicio no tiene el decorador `@Injectable` correctamente configurado o no está registrado en el sistema de inyección.

- Dependencia circular

Sucede cuando el Servicio A depende del Servicio B y el Servicio B depende del Servicio A. En general, se resuelve reorganizando la lógica para evitar ese acoplamiento.

Reactividad con RxJS, AsyncPipe, HTTP y Formularios Reactivos

Introducción

La evolución del desarrollo frontend hacia 2026 consolidó un cambio profundo en la forma de construir aplicaciones. Ya no alcanza con manipular el DOM o responder eventos de manera aislada: hoy trabajamos con flujos de datos, estados que cambian en el tiempo y múltiples fuentes asíncronas que deben coordinarse de forma predecible.

En este contexto, la reactividad dejó de ser un concepto avanzado para convertirse en una base arquitectónica. Angular adopta este enfoque combinando RxJS, HttpClient, formularios reactivos y, en versiones recientes, un modelo híbrido que integra Observables con Signals para optimizar la detección de cambios.

Esta unidad profundiza en cómo orquestar estos elementos para construir aplicaciones resilientes, mantenibles y eficientes, capaces de manejar múltiples flujos de datos sin perder claridad ni control.

1. El paradigma de la programación reactiva y RxJS

La programación reactiva es un paradigma orientado al manejo de **flujos de datos asíncronos** y a la propagación automática de cambios. En lugar de indicar paso a paso qué hacer y cuándo hacerlo (modelo imperativo), se define qué debe ocurrir cuando los datos cambian (modelo declarativo).

En Angular, esta capacidad se apoya en **RxJS** (Reactive Extensions for JavaScript), una biblioteca diseñada para trabajar con secuencias de eventos en el tiempo.

RxJS combina varios conceptos clave:

- Patrón **Observer** (suscripción a eventos).
- Patrón **Iterator** (secuencias de valores).
- Programación funcional (operadores encadenables).

El concepto central: Observable

Los observables son una parte fundamental de la programación reactiva y se utilizan comúnmente en aplicaciones Angular para gestionar flujos de datos asíncronos. Los observables proporcionan una forma elegante de trabajar con secuencias de datos que pueden cambiar con el tiempo.

¿Qué es?

Un observable es un objeto que representa una secuencia de datos asíncronos. Estos datos pueden ser eventos, resultados de solicitudes HTTP, cambios en propiedades, o cualquier otra fuente de información que cambie con el tiempo. Los observables permiten observar (escuchar) estos cambios y reaccionar a ellos de manera asincrónica.

Características claves:

- **Asincronía:** Los observables se utilizan para manejar operaciones asíncronas, lo que significa que no bloquean la ejecución del programa mientras esperan resultados.
- **Emisión de datos:** Los observables emiten datos a lo largo del tiempo. Pueden emitir varios valores en una secuencia.
- **Manejo de errores:** Los observables permiten manejar errores que pueden ocurrir durante la ejecución de una operación asíncrona.

- **Completado:** Un observable puede completarse, lo que significa que no emitirá más datos y se dará por terminado.

Usos más comunes:

- **Solicitudes HTTP:** Cuando haces una solicitud HTTP utilizando el módulo `HttpClient`, obtienes un observable como respuesta. Puedes suscribirte a este observable para recibir los datos de respuesta.
- **Manejo de eventos:** Puedes usar observables para manejar eventos del usuario, como clics de botones o entradas de formularios, utilizando la API de RxJS en Angular.
- **Actualizaciones en tiempo real:** Si trabajas con aplicaciones en tiempo real, como chat en vivo o paneles de control, los observables son útiles para recibir actualizaciones automáticas de los datos del servidor.
- **Manejo de datos reactivos:** Los observables son una parte fundamental del enfoque reactivo en Angular, que permite construir interfaces de usuario altamente responsivas y dinámicas.

En Angular, los observables se implementan utilizando RxJS (Reactive Extensions for JavaScript), una biblioteca que proporciona una amplia gama de operadores para transformar y combinar observables.

La ventaja principal de los Observables es que pueden:

- Transformarse (**map**)
- Filtrarse (**filter**)
- Combinarse (**merge**, **combineLatest**)
- Encadenarse (**switchMap**)
- Reintentarse (**retry**)
- Cancelarse (**unsubscribe**)

Esto permite tratar cualquier operación asíncrona bajo una misma interfaz conceptual, reduciendo complejidad y evitando problemas como condiciones de carrera.

Modelo mental de un flujo reactivo

Podemos representar el ciclo así:



1. Se genera una fuente (HTTP, evento, timer).
2. Se crea un Observable.
3. Se aplican operadores de transformación.
4. Un suscriptor consume el resultado.

Ventaja estratégica en aplicaciones modernas

En aplicaciones complejas, múltiples eventos pueden ocurrir simultáneamente:

- El usuario escribe en un campo.
- Se dispara una búsqueda.
- Llega una respuesta HTTP.
- Se cancela una petición anterior.
- Se actualiza la vista.

Con un modelo imperativo, esto genera código difícil de mantener. Con RxJS, se modelan estos comportamientos como flujos encadenados, donde cada transformación queda declarada.

Por ejemplo, en una búsqueda en tiempo real:

```
input$.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(texto => this.api.buscar(texto))
).subscribe(resultados => this.lista = resultados);
```

Aquí se expresa claramente:

- Esperar 300 ms.
- Ignorar valores repetidos.
- Cancelar búsquedas anteriores si llega una nueva.
- Actualizar la lista cuando llegue la respuesta.

Este tipo de composición es una de las razones por las que RxJS sigue siendo central en Angular en 2026, incluso en un entorno donde Signals conviven como mecanismo de estado local.

En síntesis, la programación reactiva no es simplemente una técnica para manejar asincronía. Es una forma de modelar aplicaciones modernas donde los datos cambian constantemente y la interfaz debe responder de manera coherente, eficiente y predecible.

Anatomía de un Observable y ciclo de vida del stream

Un Observable en RxJS no es simplemente un objeto que emite datos. Es un **contrato de ejecución** que define cómo y cuándo se producen valores, cómo se consumen y cuándo se liberan los recursos asociados. Entender su ciclo de vida es esencial para evitar fugas de memoria y comportamientos inesperados en aplicaciones Angular modernas.

En términos generales, un Observable atraviesa cuatro etapas:

1. Creación
2. Suscripción
3. Ejecución
4. Finalización o cancelación

1. Creación

La creación puede realizarse mediante funciones utilitarias como:

- **of()**: emite valores inmediatos.
- **from()**: convierte promesas, arrays u otras estructuras en flujos.
- **interval()**: emite valores periódicamente.
- **new Observable()**: definición manual personalizada.

Ejemplo simple:

```
import { interval } from 'rxjs';  
const contador$ = interval(1000);
```

Aquí se define el flujo, pero todavía no está ejecutándose.

2. Suscripción

Una vez que tienes un observable, puedes suscribirte a él utilizando el método **subscribe()**. Este método acepta uno o más argumentos de devolución de llamada (funciones) que se ejecutarán cuando el observable emita valores, errores o se complete. Por lo general, se proporcionan tres devoluciones de llamada: **next**, **error** y **complete**. Sin embargo, sólo **next** es obligatorio.

Un Observable **no hace nada hasta que alguien se suscribe**. La suscripción conecta al consumidor con el productor de datos.

```
import { Observable, interval } from 'rxjs';

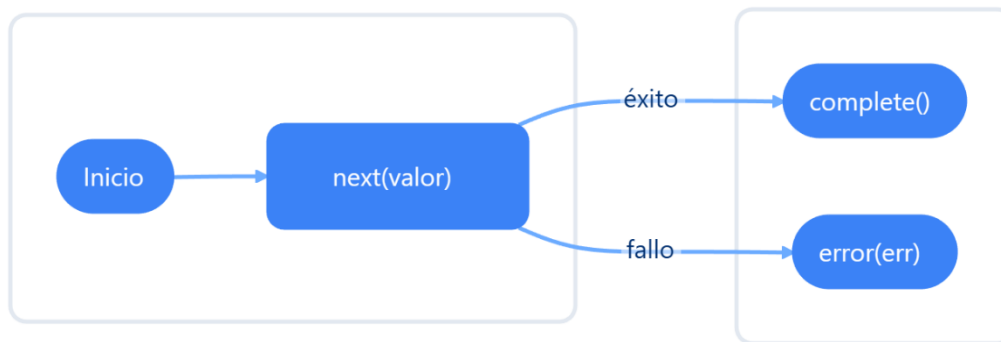
const suscripcion = intervalo.subscribe(
  (valor) : void => {
    console.log('Valor emitido', valor);
    // Realiza acciones con el valor emitido
  },
  (error) : void => {
    console.error('Error', error);
    // Realiza acciones para manejar el error
  },
  () : void => {
    console.log('Observable completado');
    // Realiza acciones cuando el observable se completa
  },
);
```

3. Ejecución

Durante su ejecución, un Observable puede emitir tres tipos de notificaciones:

1. La función **next** se ejecutará cada vez que el observable emita un valor.
2. La función **error** se ejecutará si ocurre un error en el observable.
3. La función **complete** se ejecutará cuando el observable se complete y ya no emita más valores.

Podemos representarlo así:



Una vez que ocurre **error** o **complete**, el flujo finaliza y no se emiten más valores.

4. Finalización o cancelación

La ejecución puede finalizar de tres maneras:

- El Observable llama a **complete()**.
- Se produce un **error**.
- El consumidor cancela la suscripción (**unsubscribe()**).

En Angular moderno, esta última opción ya no se maneja manualmente en la mayoría de los casos.

Gestión moderna de suscripciones en Angular 2026

En versiones anteriores era común almacenar suscripciones en propiedades y llamar a **unsubscribe()** dentro de **ngOnDestroy**. Ese enfoque sigue siendo válido, pero hoy se considera más limpio utilizar patrones declarativos que automaticen la limpieza.

Una de las soluciones actuales es el operador **takeUntilDestroyed**, integrado con el ciclo de vida del componente.

Ejemplo

```
import { Component, inject } from '@angular/core';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { TimerService } from './timer.service';

@Component({
  standalone: true,
  selector: 'app-reloj-vivo',
  template: `<p>Tiempo transcurrido: {{ segundos }}</p>`
})
export class RelojVivoComponent {

  private timerService = inject(TimerService);

  segundos = 0;

  constructor() {
    this.timerService.contador$
      .pipe(takeUntilDestroyed())
      .subscribe(val => this.segundos = val);
  }
}
```

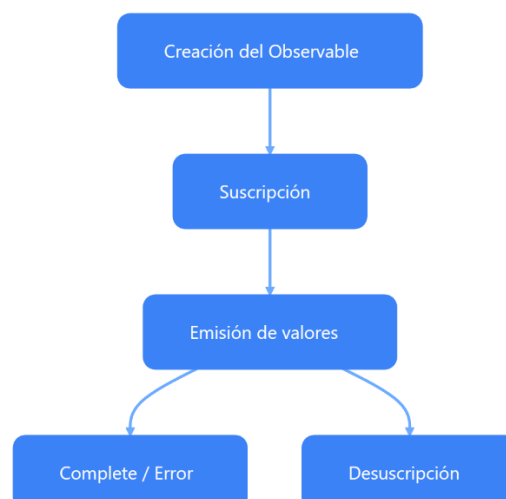
Acá sucede algo importante.

contador\$ podría estar basado en un **interval()** infinito u otro flujo continuo. En un enfoque tradicional, olvidar cancelar esa suscripción generaría una fuga de memoria.

Sin embargo, **takeUntilDestroyed()** enlaza automáticamente la suscripción al ciclo de vida del componente. Cuando el componente se destruye, el stream se cancela sin necesidad de intervención manual.

De esta manera, mantenemos un código reactivo, claro y seguro, utilizando una propiedad clásica (segundos) sin necesidad de **signal()**, pero con una gestión moderna y automática de recursos.

Diagrama del ciclo completo



Observables finitos vs infinitos

Es fundamental distinguir entre dos grandes tipos:

Observables finitos

Ejemplo: `HttpClient.get()`

- Emiten un único valor.
- Llamam automáticamente a `complete()`.
- No requieren desuscripción manual.

```
this.http.get('/api/users').subscribe(data => {  
  this.usuarios = data;  
});
```

Aquí no hay riesgo de fuga de memoria porque el flujo termina solo.

Observables infinitos

Ejemplo:

- `interval()`
- Eventos del DOM
- WebSockets
- Streams personalizados

Estos no se completan automáticamente y deben limitarse con operadores como:

- `take(1)`
- `takeWhile()`
- `takeUntil()`
- `takeUntilDestroyed()`

Ejemplo:

```
interval(1000)  
  .pipe(take(5))  
  .subscribe();
```

Aquí el flujo se corta luego de 5 emisiones.

Buenas prácticas actuales

En Angular 2026, el enfoque profesional incluye:

- Preferir **AsyncPipe** en templates cuando sea posible.
- Usar **takeUntilDestroyed()** en componentes.
- Evitar suscripciones anidadas.
- Encadenar operadores declarativos en lugar de lógica imperativa.
- Mantener la lógica reactiva dentro de servicios.

Comprender el ciclo de vida completo del Observable permite construir aplicaciones más robustas, optimizadas y alineadas con las prácticas modernas del ecosistema Angular.

Operadores RxJS: el motor de la transformación

Los operadores son funciones que permiten transformar, filtrar o combinar los datos que circulan dentro de un **pipe**. Son el núcleo de la programación reactiva: gracias a ellos, un flujo simple puede convertirse en una lógica de negocio clara, expresiva y eficiente.

En Angular moderno, el uso correcto de operadores marca la diferencia entre un código difícil de mantener y uno declarativo, predecible y escalable.

Categorización de operadores esenciales

Para facilitar su comprensión, podemos agruparlos según el tipo de problema que resuelven:

Categoría	Operadores Clave	Propósito y Aplicación
Creación	of, from, interval, defer	Convertir datos estáticos o promesas en flujos reactivos.
Transformación	map, scan, pluck	Modificar la forma o el valor de los datos emitidos.
Filtrado	filter, debounceTime, distinctUntilChanged	Descartar emisiones innecesarias o ruidosas basándose en el tiempo o valor.
Combinación	combineLatest, forkJoin, merge	Orquestar múltiples fuentes para producir un resultado unificado.
Aplanamiento	switchMap, mergeMap, concatMap	Gestionar observables anidados (un stream que dispara otro stream).
Utilidad	tap, delay, timeout	Ejecutar efectos secundarios o manipular la temporización del flujo.

Operadores de aplanamiento: un punto clave

Cuando un Observable dispara otro (por ejemplo, una búsqueda que genera una petición HTTP), se produce un caso de “observable dentro de observable”. Para manejar esto se utilizan operadores de aplanamiento.

Entre ellos, **switchMap** es uno de los más importantes en interfaces modernas.

¿Qué hace switchMap?

- Cancela la suscripción anterior.
- Se queda únicamente con la última emisión.
- Evita que respuestas viejas sobrescriban resultados nuevos.

Esto es fundamental en buscadores o inputs con escritura rápida.

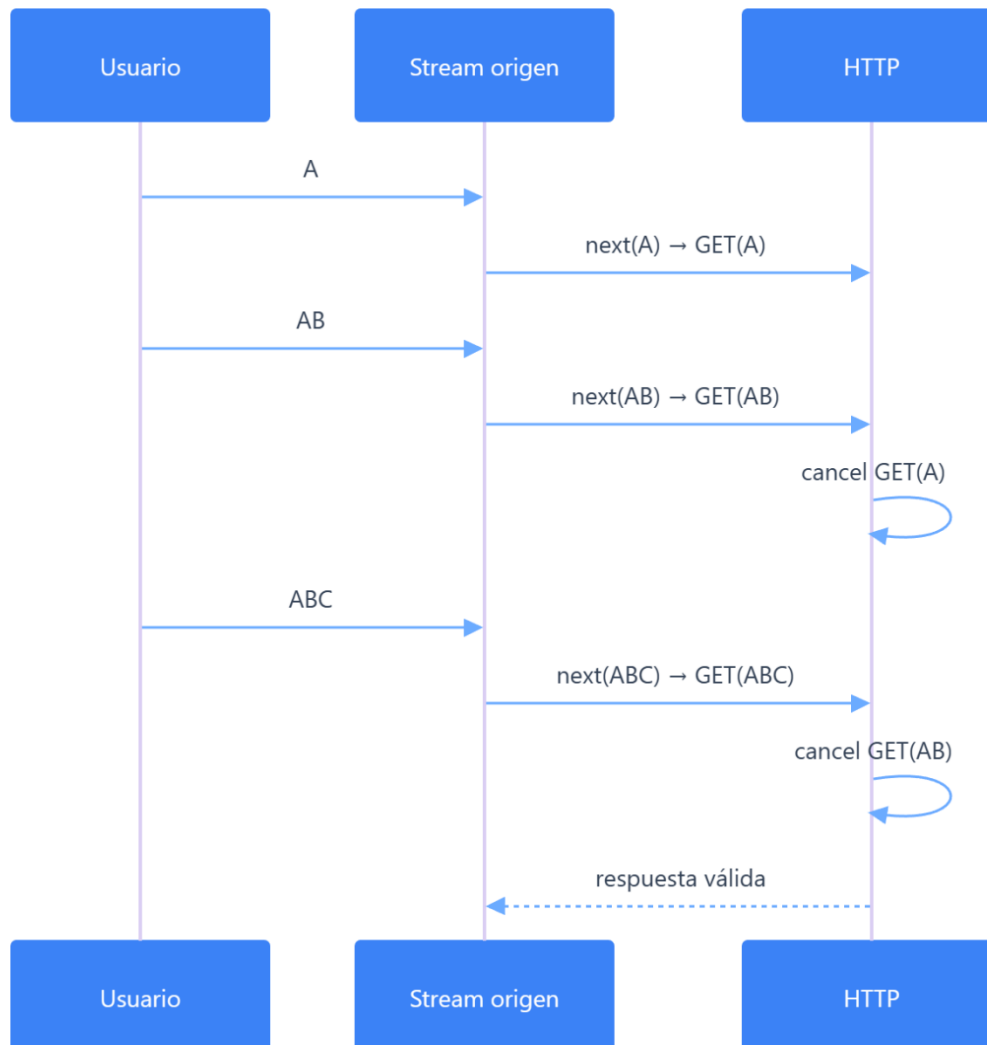
Ejemplo conceptual con switchMap

```
input$  
  .pipe(  
    debounceTime(300),  
    distinctUntilChanged(),  
    switchMap(texto => this.http.get(`/api?q=${texto}`))  
  )  
  .subscribe(resultados => this.lista = resultados);
```

Aquí ocurre lo siguiente:

1. Se espera 300 ms después de que el usuario deje de escribir.
2. Se ignoran valores repetidos.
3. Se cancela cualquier petición anterior si llega una nueva.
4. Solo la última respuesta actualiza la interfaz.

Esquema simplificado del comportamiento de switchMap



Sin **switchMap**, una respuesta lenta de **GET(A)** podría llegar después que **GET(ABC)** y sobrescribir los resultados correctos.

Con **switchMap**, esto no ocurre.

HttpClient

Angular utiliza **HttpClient** junto con Observables de la biblioteca RxJS. A diferencia de las Promesas tradicionales de JavaScript, los Observables ofrecen más flexibilidad:

- Permiten manejar múltiples emisiones de datos.
- Pueden cancelarse.

- Se integran con operadores funcionales como **map**, **filter**, **switchMap**, entre otros.
- Son la base del enfoque reactivo del framework.

Configuración

Antes de utilizar **HttpClient** en su aplicación Angular, es imprescindible configurarlo correctamente mediante **inyección de dependencia**. Esta configuración asegura que los servicios que requieren acceso a la red puedan comunicarse con API externas de manera efectiva y segura.

Se ofrece a través de la función auxiliar `provideHttpClient()`, que permite a Angular gestionar las solicitudes HTTP de manera eficiente. Esta función generalmente se incluye en la configuración de los proveedores de la aplicación, que se define en un archivo de configuración como `app.config.ts`.

Funcionalidades configurables

La API moderna permite extender el comportamiento mediante funciones **with...**:

withFetch()

Utiliza la API `fetch` nativa en lugar de `XMLHttpRequest`, alineándose mejor con estándares actuales y entornos híbridos (como SSR).

withInterceptors([...])

Permite registrar interceptores como funciones puras, lo que simplifica su composición y mantenimiento.

withInterceptorsFromDi()

Mantiene compatibilidad con interceptores tradicionales basados en clases e inyección de dependencias.

withJsonpSupport()

Habilita soporte JSONP para casos heredados donde el backend lo requiera.

En versiones recientes del framework, **HttpClient** suele venir habilitado en configuraciones iniciales para facilitar el inicio. Sin embargo, en aplicaciones productivas se recomienda configurarlo explícitamente, especialmente cuando se necesitan interceptores de autenticación, manejo centralizado de errores o logging.

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(),
    // Aquí puedes agregar otros proveedores que necesite tu aplicación
  ]
};
```

Injectar HttpClient en tu servicio o componente:

Para utilizar HttpClient en un servicio o componente, es necesario inyectar y en angular se puede hacer mediante el constructor.

```
import { HttpClient } from '@angular/common/http'

no usages
@Injectable({
  providedIn: root
})
export class NuevoService {
  no usages
  constructor(private http: HttpClient) { }

  no usages
  obtenerDatos(): Observable<Product> {
    return this.http.get<Product>({
      url: 'https://api.example.com/data'
    });
  }
}
```

Manejar la respuesta:

Para obtener la respuesta de la solicitud HTTP desde el componente es necesario suscribirse a dicho método el cual una vez que la API responda emitirá la respuesta. Ejemplo:

Injectar servicio en componente y realizar la llamada:

```
import { NuevoService } from './nuevo.service.ts';

no usages
@Component({
  selector: 'app-prueba',
  templateUrl: './app-prueba.component.html'
})
export class PruebaComponent implements OnInit {
  listado: Product[] = [];
  no usages
  constructor(private servicio: NuevoService) { }

  no usages
  ngOnInit(): void {
    this.servicio.obtenerDatos().subscribe({
      next: (response: Product[]) : void => {
        this.listado = response;
        // Aquí puedes realizar cualquier otra acción con los datos recibidos
      },
      error: (error: Error): void => {
        console.error(error);
        // Aquí puedes realizar el manejo del error como se desee.
        // en este caso se imprime un mensaje en la consola del navegador
      }
    });
  }
}
```

```
import { NuevoService } from './nuevo.service.ts';

no usages
@Component({
  selector: 'app-prueba',
  templateUrl: './app-prueba.component.html'
})
export class PruebaComponent implements OnInit {
  listado: Product[] = [];
  no usages
  constructor(private servicio: NuevoService) { }

  no usages
  ngOnInit(): void {
    this.servicio.obtenerDatos().subscribe({
      next: (response: Product[]) : void => {
        this.listado = response;
        // Aquí puedes realizar cualquier otra acción con los datos recibidos
      },
      error: (error: Error): void => {
        console.error(error);
        // Aquí puedes realizar el manejo del error como se desee.
        // en este caso se imprime un mensaje en la consola del navegador
      }
    });
  }
}
```

Como se puede ver existen dos alternativas para el manejo de las respuestas al suscribirse a un Observable. La primera es la utilizada en el ejemplo de observables utilizando un intervalo y la segunda es una forma simplificada que no requiere la declaración de las funciones **callback** (next, error, complete).

Otro punto interesante es que no es obligatorio implementar el manejo de los errores o la completitud de la notificación emitida por el Observable, aunque es una buena práctica manejar los escenarios de errores para ofrecer un buen feedback a los usuarios de nuestras aplicaciones.

Una vez que se llama al servicio que hace la llamada a la API en caso de obtener una respuesta satisfactoria se carga el dicho resultado en una propiedad existente en el componente, en el ejemplo se asigna el arreglo de Productos proveniente de la API a la propiedad listado dentro del componente.

Ejemplo conceptual de petición

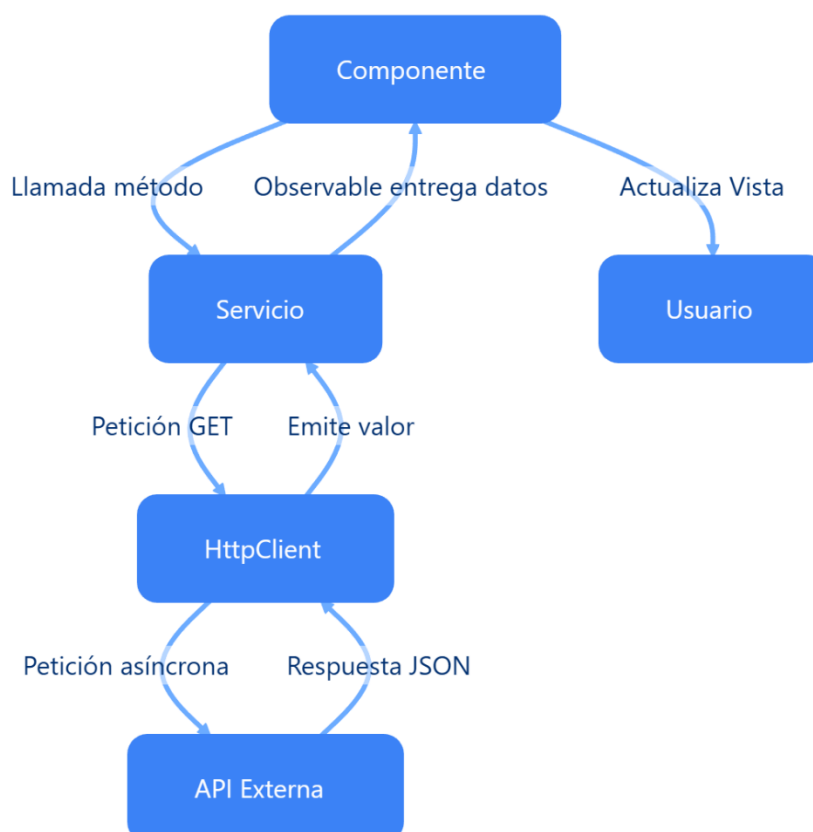
En un servicio:

```
obtenerUsuarios() {
  return this.http.get('https://api.ejemplo.com/users');
}
```

En el componente:

```
this.servicio.obtenerUsuarios().subscribe(data => {  
  this.usuarios = data;  
});
```

Diagrama: Flujo de datos HTTP



En aplicaciones más complejas, suele combinarse con **operadores** de **RxJS** o con el **pipe async** en la plantilla para manejar la suscripción automáticamente.

Buenas prácticas actuales

- En Angular moderno (v16+), es común:
- Tipar las respuestas HTTP con genéricos, por ejemplo:
- **this.http.get<Usuario[]>('...')**
- Manejar errores con `catchError`.
- Encapsular la lógica HTTP dentro de servicios.
- Utilizar interceptores para agregar tokens JWT o manejar errores globales.
- Esto mejora la escalabilidad y la mantenibilidad del proyecto.

Errores comunes y diagnóstico

- Olvidar `subscribe()`
- Si no hay suscripción, la petición no se envía. Si el código parece no hacer nada, conviene verificar este punto.
- Errores de CORS
- No es un problema de Angular. El navegador bloquea solicitudes si el servidor no permite el origen (por ejemplo, `localhost:4200`). La solución está en configurar correctamente el backend.
- No manejar errores
- Si el backend responde con error y no se implementa manejo de errores, la aplicación puede fallar silenciosamente o romper la experiencia de usuario.

Formularios en Angular

Un formulario constituye una estructura esencial en el desarrollo web moderno, actuando como el puente primario de comunicación que permite a los usuarios ingresar, validar y enviar información a una aplicación. A través de una interfaz gráfica compuesta por campos de entrada, etiquetas y controles, los formularios sirven como el principal punto de interacción para la recopilación de datos, la realización de acciones específicas (como una compra o un inicio de sesión) y el envío seguro de información al servidor. Su correcto diseño e implementación son cruciales para la usabilidad, la seguridad y la funcionalidad de cualquier aplicación. Componentes Clave de un Formulario Web

Todo formulario web, independientemente del *framework* (como Angular) o la tecnología utilizada, se construye sobre un conjunto de elementos fundamentales que garantizan su operatividad:

- 1. Campos de Entrada (Input Fields):** Son la materia prima del formulario, los elementos donde el usuario interactúa directamente para proveer datos.
 - **Tipos de Datos Simples:** Incluyen campos para **Texto Simple** (`<input type="text">`), **Áreas de Texto** (`<textarea>`) para información alfanumérica extensa, y **Contraseñas** (`<input type="password">`).
 - **Controles de Selección:** Elementos como **casillas de verificación** (*checkboxes*), **botones de radio** (*radio buttons*) para selecciones excluyentes, y **campos de selección** (*select/dropdowns*) para elegir opciones de una lista predefinida.
 - **Campos Tipados Específicos:** El estándar HTML5 provee tipos que facilitan la validación y la interacción del usuario, como `email`, `tel`, `url`, `date`, `number`, `range`, entre otros.

2. **Etiquetas (Labels):** Son descripciones textuales que están **asociadas semánticamente** a un campo de entrada. Su función es indispensable para la **usabilidad (UX)** y la **accesibilidad (A11y)**, utilizando el atributo **for** en el *label* para vincularlo al **id** del *input*. Indican claramente al usuario qué tipo de información se espera en cada campo, lo cual es vital para usuarios con lectores de pantalla.
3. **Botones (Buttons):** Elementos interactivos que permiten ejecutar flujos de trabajo.
 - **Envío (Submit):** El botón esencial, encargado de procesar la información del formulario e iniciar la comunicación con el *backend*.
 - **Restablecer (Reset):** Limpia todos los campos del formulario a sus valores iniciales.
 - **Cancelar:** Aborta la operación del formulario, típicamente navegando fuera de la vista.
4. **Validación del Lado del Cliente (Client-Side Validation):** Es el conjunto de reglas y comprobaciones que garantizan la calidad, el formato correcto y la integridad de los datos **antes de que sean enviados** al servidor.
 - **Requeridos:** Comprobar que se completen **campos obligatorios**.
 - **Formato y Tipo:** Verificar que los datos sean del **tipo correcto** (ej. un número en un campo numérico) y que respeten formatos específicos (ej. estructura de un correo electrónico).
 - **Restricciones de Datos:** Asegurar que los datos cumplan con **criterios específicos** como longitud mínima/máxima o patrones de expresión regular (regex).
5. **Acciones de Envío y Servidor:** Una vez completado y validado en el cliente, la acción de envío (típicamente a través de un *HTTP POST* o *PUT*) desencadena una operación crítica en el *backend*, como **guardar datos** en una base de datos, **enviar notificaciones** o **iniciar un proceso de negocio** (ej. la creación de un pedido).
6. **Estilo y Diseño (UI/UX):** El diseño visual y la estructura de interacción del formulario son determinantes para la **Experiencia de Usuario (UX)**. Un diseño claro, con mensajes de error precisos y un flujo lógico, facilita la comprensión, reduce la frustración del usuario y disminuye la tasa de errores y abandono.

Los formularios son omnipresentes, utilizándose para el **registro e inicio de sesión** (autenticación y autorización), la **publicación de contenido** (comentarios, *posts*), la **configuración de perfiles** y, fundamentalmente, en los procesos centrales del **comercio electrónico** (carritos de compra y *checkouts*).

Tipos de Formularios en Angular: Enfoques para el Desarrollo

Angular proporciona dos arquitecturas robustas y distintas para la construcción, gestión y validación de formularios. La elección entre ambos depende de la complejidad y la escalabilidad requerida por la aplicación.

- Template-Driven Forms (Formularios Basados en Plantillas)
- Reactive Forms (Formularios Reactivos / Enfoque Model-Driven)

1. Template-Driven Forms (Formularios Basados en Plantillas)

Este enfoque utiliza la **plantilla HTML** como la fuente principal de verdad para el formulario. La mayor parte de la lógica, incluyendo la estructura y la definición de las validaciones, se inyecta **directamente en el markup HTML** a través del uso extensivo de **directivas** específicas de Angular. **Características Operativas Clave**

- **Definición en el Template:** La estructura y el modelo de datos son inferidos por Angular a partir del HTML.
- **Ideal para Casos Sencillos:** Se implementan de forma muy rápida y son adecuados para formularios básicos, como un simple formulario de contacto o configuraciones sencillas que no requieren una validación dinámica compleja.
- **Dependencia de Directivas:** La funcionalidad se basa en la aplicación de directivas como `ngForm`, `ngModel`, y `ngSubmit`.
- **Modelo de Datos Implícito:** El estado del formulario y de sus controles es gestionado internamente por Angular, con menos control explícito desde el componente TypeScript.

Directivas Fundamentales y Sintaxis

Directiva	Propósito Central	Detalle Clave
<code>ngForm</code>	Activa el rastreo del formulario completo por Angular.	Se aplica a la etiqueta <code><form></code> . Habilita el seguimiento del estado general (<code>valid</code> , <code>dirty</code> , etc.) de todos los controles internos.
<code>ngModel</code>	Implementa la vinculación bidireccional (<i>Two-Way Binding</i>).	Conecta un elemento de entrada (<code><input></code>) con una propiedad del componente TypeScript. Requiere importar <code>FormsModule</code> . Sintaxis: <code>[(ngModel)]="propiedadComponente"</code> .
<code>ngSubmit</code>	Captura el evento de envío del formulario.	Previene el comportamiento nativo de recarga del navegador. Se asocia a una función del componente para manejar el envío: <code>(ngSubmit)="funcionManejo(\$event)"</code> .
<code>name</code> Attribute	Identificador Único (Esencial para TDF).	Obligatorio para cualquier elemento de entrada que use <code>ngModel</code> dentro de un <code>ngForm</code> . Angular lo utiliza para registrar el control y su estado dentro del objeto <code>ngForm</code> global.

Acceso a Controles y Propiedades

Para interactuar con el estado de un control individual desde el *template* (ej. para mostrar un mensaje de error), se utiliza una **variable de referencia de plantilla (#)**:

```
<input type="text" [(ngModel)]="usuario.nombre" name="nombre" required
#nombreControl="ngModel">
<div *ngIf="nombreControl.invalid && nombreControl.touched">
  El nombre es obligatorio.
</div>
```

Validaciones en TDF

Los *Template-Driven Forms* dependen en gran medida de los **atributos de validación nativos de HTML5**. Angular lee y traduce estos atributos en reglas de validación que expone a través de la instancia `ngModel`.

Atributo HTML	Propósito Específico
<code>required</code>	Marca el campo como obligatorio.
<code>minlength</code> , <code>maxlength</code>	Controla la cantidad mínima y máxima de caracteres permitidos.
<code>pattern</code>	Permite especificar una Expresión Regular (regex) para validar el formato del valor.
<code>min</code> , <code>max</code>	Controla el valor mínimo y máximo para campos numéricos o de fecha.

Agrupación de Datos: `ngModelGroup`

Para modelar objetos anidados o estructuras de datos jerárquicas (ej. `usuario.direccion.calle`), se emplea la directiva `ngModelGroup`. Esta directiva crea un **ModelGroup**, que actúa como un subformulario que agrupa los controles internos, mapeándolos a una propiedad de objeto anidado en el modelo del componente.

Manejo de Estados de Formularios y Controles

Una de las mayores fortalezas de Angular es su capacidad para **rastrear automáticamente el estado de cada control y del formulario completo**, un mecanismo vital para proveer *feedback* instantáneo al usuario.

Propiedades de Estado Clave

Estas propiedades existen tanto en los controles individuales (`FormControl` o `ngModel`) como en el formulario completo (`FormGroup` o `ngForm`):

Propiedad	Descripción Detallada	Uso Típico
<code>valid</code>	<code>true</code> si el control/formulario cumple con todas las reglas de validación.	Habilitar el botón de envío (<code><button [disabled]="!formulario.valid"></code>).
<code>invalid</code>	<code>true</code> si existe al menos un error de validación.	Mostrar mensajes de error.
<code>pristine</code>	<code>true</code> si el usuario NO ha modificado el valor inicial.	Ocultar mensajes de error hasta que el usuario intente modificar algo.
<code>dirty</code>	<code>true</code> si el usuario HA modificado el valor inicial.	Mostrar una alerta antes de que el usuario abandone la página sin guardar.
<code>untouched</code>	<code>true</code> si el usuario NO ha interactuado con el control (nunca ha recibido ni perdido el foco).	Validaciones que no deben mostrarse inmediatamente al cargar el formulario.
<code>touched</code>	<code>true</code> si el usuario HA interactuado con el control (le dio foco y luego lo perdió, o <i>blur</i>).	El estado ideal para mostrar un mensaje de error: <code>invalid && touched</code> .

Clases CSS Automáticas

Angular facilita la estilización dinámica al inyectar automáticamente clases CSS a los elementos del formulario (`<form>`, `<input>`, `<select>`, etc.) que reflejan su estado.

Clase CSS	Estado Correspondiente	Aplicación de Estilo
<code>ng-valid</code>	<code>valid: true</code>	Borde verde.
<code>ng-invalid</code>	<code>invalid: true</code>	Borde rojo o ícono de advertencia.
<code>ng-pristine</code>	<code>pristine: true</code>	Estado inicial sin cambios.
<code>ng-dirty</code>	<code>dirty: true</code>	El campo ha sido modificado.
<code>ng-untouched</code>	<code>untouched: true</code>	No se ha interactuado con el campo.
<code>ng-touched</code>	<code>touched: true</code>	Se ha interactuado con el campo.

2. Reactive Forms: La Arquitectura Model-Driven de Angular

Los Formularios Reactivos representan el enfoque Model-Driven (Impulsado por el Modelo) de Angular, consolidándose como el método estándar y recomendado para el desarrollo de aplicaciones profesionales, complejas, robustas y altamente escalables. En esta poderosa arquitectura, la responsabilidad de gestionar la lógica, la estructura, la inicialización y las validaciones del formulario se define de manera explícita, declarativa y

centralizada en la clase del componente TypeScript. Este componente actúa como la única *fuentes de verdad* y el modelo de datos reactivo del formulario.

Superando las Limitaciones del Enfoque Basado en Plantilla

Si bien el enfoque basado en plantillas (Template-Driven Forms) ofrece una curva de aprendizaje inicial más suave, sus limitaciones se vuelven críticas a medida que la aplicación crece en complejidad:

- **Lógica Acoplada y Dispersa:** La lógica de validación, las reglas de negocio y la estructura se encuentran intrínsecamente **acopladas y mezcladas con la plantilla HTML**. Esto dispersa el código, compromete la visibilidad de las reglas y hace que el mantenimiento, la depuración y la refactorización sean considerablemente más difíciles.
- **Dificultad en Pruebas Unitarias (Testing):** Es notoriamente complejo realizar **pruebas unitarias de caja blanca** de manera efectiva y aislada sobre la lógica de validación sin tener que cargar y simular el Document Object Model (DOM), lo que ralentiza el desarrollo y las canalizaciones de CI/CD.
- **Control Programático Limitado:** La necesidad de manipular dinámicamente los valores, el estado (*valid*, *dirty*, *touched*, etc.) y las validaciones del formulario desde la clase TypeScript se vuelve **engorrosa y menos intuitiva**.

El Paradigma Reactivo: La Sinergia con RxJS

La denominación "Reactivos" se debe a que su diseño fundamental está intrínsecamente ligado al paradigma de la **programación reactiva** y hace un uso intensivo de la biblioteca **Observables (RxJS)**, lo que permite un manejo asíncrono y en tiempo real de los datos:

1. **Estado Programático y Observable (La Fuente de Verdad):** El modelo completo del formulario se define y reside exclusivamente en TypeScript, lo que lo convierte en un objeto manipulable programáticamente.
2. **Emisiones de Datos Constantes:** Cada interacción del usuario (ingreso de texto, selección, etc.) o cambio programático de valor en un control no es un simple evento; en cambio, **genera una emisión observable continua** a través de las propiedades clave: `valueChanges` (para cambios de valor) y `statusChanges` (para cambios en el estado de validez).
3. **Reacción, Suscripción y Lógica Dinámica:** El desarrollador tiene la capacidad de **suscribirse (subscribe)** a estas emisiones de datos. Esto habilita la creación de lógica de negocio sumamente avanzada, permitiendo **reaccionar**

programáticamente a los cambios en tiempo real. Por ejemplo: implementar validaciones condicionales que dependen del valor de otros campos, manipular flujos de datos asíncronos (como el debounce para búsquedas) o activar lógica de negocio compleja solo cuando el formulario alcanza un estado específico.

Gestión de Validaciones: Sincrónicas y Asíncronas

Una de las fortalezas clave del enfoque reactivo es la manera desacoplada y potente en que gestiona la validación de datos. Las validaciones se definen en el modelo TypeScript, permitiendo dos tipos de ejecución:

Validaciones Sincrónicas (Synchronous Validators)

Son funciones puras que se ejecutan inmediatamente cuando cambia el valor de un control.

- **Implementación:** Se proporcionan como el segundo argumento al instanciar un `FormControl`.
- **Propósito:** Se utilizan para reglas de validación simples e instantáneas que no requieren operaciones demoradas, como verificar si un campo está vacío, si cumple con un formato de expresión regular, o si está dentro de un rango de longitud.
- **Ejemplos Comunes:** `Validators.required`, `Validators.minLength()`, `Validators.email`, `Validators.pattern()`.
- **Retorno:** Devuelven un objeto de error (ej. `{ 'required': true }`) si la validación falla, o `null` si es válida.

```
// Ejemplo de validadores sincrónicos en la definición de un FormControl
nombre: new FormControl('', [
  Validators.required,
  Validators.minLength(3) // Debe tener al menos 3 caracteres
]),
```

Validaciones Asíncronas (Asynchronous Validators)

Son funciones que se ejecutan después de las validaciones sincrónicas y están diseñadas para manejar validaciones que requieren un tiempo de espera, como una petición HTTP a un servidor.

- **Implementación:** Se proporcionan como el tercer argumento al instanciar un `FormControl`.
- **Propósito:** Son esenciales para comprobaciones que necesitan acceder a recursos externos, como la verificación de la unicidad de un nombre de usuario o un correo electrónico en una base de datos.

- **Manejo de Estado:** Mientras se está ejecutando la validación asíncrona, el `FormControl` adopta el estado `pending`, lo que permite mostrar una interfaz de carga al usuario.
- **Retorno:** Devuelven un `Promise<ValidationErrors | null>` o un `Observable<ValidationErrors | null>`. El Observable es la forma recomendada, a menudo combinado con operadores de RxJS como `debounceTime()` y `distinctUntilChanged()` para optimizar las peticiones y evitar validaciones innecesarias.

```
// Ejemplo de un validador asíncrono personalizado para comprobar unicidad
email: new FormControl('', [Validators.required, Validators.email], [
  this.userService.checkEmailUniqueness.bind(this) // Validador Asíncrono
]),
```

Donde `checkEmailUniqueness` es un método que devuelve un Observable que emite `null` o un error.

Piezas Fundamentales de la Arquitectura Reactiva (La Jerarquía del Modelo)

Los formularios reactivos se construyen mediante la instanciación de un conjunto de clases principales proporcionadas por Angular, que establecen la jerarquía y la estructura del modelo en TypeScript:

Clase	Representación en la UI	Rol y Responsabilidades Clave
<code>FormControl</code>	El campo de entrada atómico (ej. <code><input></code> , <code><select></code> , <code><textarea></code>).	Es la unidad más básica. Gestiona el valor actual del campo, el estado de control individual (<code>valid</code> , <code>dirty</code> , <code>touched</code> , <code>pending</code> , etc.) y la colección de funciones validadoras sincrónicas y asíncronas asociadas.
<code>FormGroup</code>	La estructura del formulario completo o un subgrupo anidado de controles.	Es un contenedor fundamental que agrupa una colección de instancias de <code>FormControl</code> y/o otros <code>FormGroup</code> . La validez de un <code>FormGroup</code> se calcula como la suma lógica (AND) de la validez de todos sus controles internos, lo que simplifica la validación del formulario completo.
<code>FormArray</code>	Una lista o colección dinámica y ordenada de controles.	Se utiliza específicamente para modelar arrays de datos donde la cantidad de entradas es variable y puede ser modificada por el usuario (ej. una lista de <i>emails</i> , <i>hobbies</i> , o detalles de contacto). Puede contener tanto instancias de <code>FormControl</code> como de <code>FormGroup</code> .

Flujo de Trabajo Detallado (Model-Driven)

El proceso para implementar un formulario reactivo sigue un patrón claro y predecible:

1. Configuración del Módulo

Es un requisito indispensable importar el **ReactiveFormsModule** en el módulo (**NgModule**) donde se declarará el componente que contendrá el formulario.

Definición de la Estructura (TypeScript - La Fuente de Verdad)

La estructura del formulario, que incluye la jerarquía, los valores iniciales y los validadores, se define mediante la instanciación de las clases en el componente.

```
import { FormGroup, FormControl, Validators } from '@angular/forms';
// ... dentro de la clase del componente

formularioLogin = new FormGroup({
  // Campo 'nombre'
  nombre: new FormControl('', [
    Validators.required,
    Validators.minLength(3)
  ]),

  // Campo 'email' con valor inicial y múltiples validadores
  email: new FormControl('contacto@ejemplo.com', [
    Validators.required,
    Validators.email
  ]),

  // Ejemplo de grupo anidado (FormGroup)
  credenciales: new FormGroup({
    usuario: new FormControl(''),
    contraseña: new FormControl('')
  })
});
```

2. Conexión al Template (HTML - El "Binding")

El *binding* en el HTML es puramente declarativo y establece la conexión entre la estructura del modelo TypeScript y los elementos de la interfaz de usuario, sin albergar la lógica.

- **[formGroup]**: Vincula el elemento `<form>` del HTML con la instancia del **FormGroup** definida en el TS.
- **formControlName**: Vincula cada campo de entrada individual (`<input>`, etc.) con su correspondiente **FormControl** dentro del **FormGroup**.

- **formGroupName / formArrayName:** Se utilizan para vincular los grupos anidados o las colecciones dinámicas.

```
<form [formGroup]="formularioLogin" (ngSubmit)="enviarDatos()">

  <input type="text" formControlName="nombre">

  <input type="email" formControlName="email">

  <div *ngIf="!formularioLogin.get('email')?.valid &&
formularioLogin.get('email')?.touched">
    El email no es válido o es requerido.
  </div>

  <button type="submit" [disabled]="!formularioLogin.valid">
    Enviar
  </button>
</form>
```

Ventajas Definitivas del Enfoque Reactivo

La adopción de Formularios Reactivos ofrece beneficios arquitectónicos sustanciales:

- **Estructura Explícita, Fuerte Tipado y Mantenibilidad Superior:** El modelo del formulario es claro, bien definido y reside por completo en TypeScript. Esto facilita la legibilidad del código, el mantenimiento a largo plazo y permite aprovechar el **tipado fuerte** de TypeScript, mejorando la detección de errores en tiempo de compilación.
- **Facilidad de Pruebas Unitarias Desacopladas:** Dado que toda la lógica de validación reside en clases TypeScript puras, está completamente **desacoplada del DOM**. Esto permite realizar pruebas unitarias rápidas, aisladas y eficientes sobre el modelo del formulario sin necesidad de simular el navegador.
- **Integración Poderosa y Nativa con RxJS:** La base observable permite construir lógica reactiva sumamente avanzada y dinámica:
 - **Validaciones Asíncronas (Clave):** Permite implementar validaciones que requieren una operación asíncrona (ej. una llamada al servidor para comprobar la disponibilidad de un nombre de usuario en tiempo real) sin bloquear la interfaz.
 - **Control de Flujo:** Deshabilitar campos, cambiar validadores o establecer valores automáticamente basados en las emisiones de valor de otro campo.
 - **Manejo de Debounce y Throttling** para optimizar las peticiones a la API.

- **Control Programático Completo y API Rica:** El desarrollador tiene un control total sobre el ciclo de vida del formulario. Los métodos clave como `setValue()`, `patchValue()` (actualización parcial), `reset()`, `disable()`, y `setValidators()` permiten modificar valores, estados y validadores de forma programática, facilitando la implementación de complejas experiencias de usuario (UX).

En conclusión, los Formularios Reactivos permiten modelar el formulario como una **fente de datos reactiva, fuertemente controlada y aislada de la presentación**, alineándose de manera perfecta con el paradigma de programación moderna y las robustas capacidades de manipulación de datos que ofrece el *framework* Angular.

Resumen y Checklist

Angular en 2026 se posiciona como una plataforma madura, optimizada y orientada a la claridad arquitectónica. La evolución hacia componentes standalone y las mejoras continuas en rendimiento y herramientas consolidan un framework más predecible y eficiente.

A lo largo del recorrido se trabajó sobre los pilares fundamentales:

- Creación y estructura de proyectos con Angular CLI.
- Componentes standalone como unidad central de la interfaz.
- Binding unidireccional y bidireccional para conectar datos con la vista.
- Directivas y nuevo control flow optimizado por el compilador.
- Pipes para transformar datos en la presentación.
- Routing para navegación entre vistas sin recargar la página.
- Comunicación entre componentes con `@Input`, `@Output` y servicios.
- Uso de TypeScript como base tipada.
- Servicios e inyección de dependencias para organizar la lógica.
- Manejo de asincronismo con `HttpClient` y `Observables`.

El enfoque actual reduce código repetitivo, elimina la necesidad de `NgModules` en la mayoría de los casos y mejora la experiencia tanto en desarrollo como en producción.

Recursos Adicionales

- **Componentes**

<https://angular.dev/guide/components>

<https://angular.dev/guide/components/lifecycle>

- **Pipes**

<https://angular.dev/guide/pipes>

<https://angular.dev/api/common/DatePipe>

<https://angular.dev/api/common/UpperCasePipe>

<https://angular.dev/api/common/CurrencyPipe>

<https://angular.dev/api/common/DecimalPipe>

- **Directivas**

<https://angular.dev/guide/directives>

<https://angular.dev/guide/directives#built-in-attribute-directives>

<https://angular.dev/guide/directives#built-in-structural-directives>

<https://angular.dev/guide/directives#built-in-attribute-directives>

<https://angular.dev/guide/components/content-projection#>

<https://angular.dev/api/core/ng-container#>

- **Inyección de dependencia**

<https://angular.dev/guide/di>

<https://angular.dev/guide/di/dependency-injection>

- **Routing**

<https://angular.dev/guide/routing>

<https://angular.dev/guide/routing/common-router-tasks>

<https://angular.dev/api/router/provideRouter>

<https://angular.dev/guide/routing/routing-with-urlmatcher>

- **Angular 20 y 21**

<https://blog.angular.dev/announcing-angular-v20-b5c9c06cf301>

<https://blog.angular.dev/announcing-angular-v21-57946c34f14b>

<https://angular.dev/guide/testing>

Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera:

Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.